

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «СЕВЕРО-КАВКАЗСКИЙ
ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

(СКФУ)

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ

**Разработка системы управления профилем пользователя и
групповыми чатами для мессенджера на базе протокола Matrix**

Выполнил: Иванов Дмитрий Романович
3 курс, группа ПИЖ-б-о-23-2
09.03.04 «Программная инженерия»
очная форма обучения

Проверил: Ассистент департамента цифровых, робототехнических систем
и электроники
Потапов И. Р.

Отчёт защищён с оценкой _____

Дата защиты _____ 2026
Ставрополь 2026

РЕФЕРАТ

Отчёт 58 с., 24 рис., 11 табл., 3 прил.

СОЦИАЛЬНАЯ СЕТЬ, МЕССЕНДЖЕР, ПРОТОКОЛ MATRIX, JAVA, SPRING BOOT, REACT, REST API, РСМЕF, ГЕКСАГОНАЛЬНАЯ АРХИТЕКТУРА

В пояснительной записке описана разработка системы управления профилем пользователя и групповыми чатами для мессенджера на базе протокола Matrix. В аналитической части проведён анализ предметной области, бизнес-процессов, конкурентов и обоснована необходимость разработки. В проектной части представлены модель требований, модель предметной области, архитектурное проектирование на основе паттерна РСМЕF с адаптацией под гексагональные принципы, проектирование базы данных и детальное проектирование с применением паттернов GoF. В реализационной части описана разработка бизнес-логики, пользовательского интерфейса на React, REST API с применением OpenAPI, системы безопасности и транзакций. В разделе тестирования приведены результаты модульного, интеграционного и системного тестирования. В разделе развёртывания описана контейнеризация с использованием Docker и деплой в Kubernetes через Helm. В разделе управления проектом представлены WBS, диаграмма Ганта, оценка трудозатрат по модели COCOMO и управление рисками.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	6
1.1	Актуальность	6
1.2	Цель и задачи	7
1.3	Объект и предмет исследования	8
2	1. АНАЛИТИЧЕСКАЯ ЧАСТЬ	9
2.1	1.1. Описание предметной области	9
2.2	1.2. Анализ бизнес-процессов (IDEF0)	9
2.3	1.3. SWOT-анализ текущего состояния	10
2.4	1.4. Анализ аналогов и конкурентов	11
2.5	1.5. Обоснование необходимости разработки	12
2.6	1.6. Экономическое обоснование (ROI)	12
3	2. ПРОЕКТНАЯ ЧАСТЬ	14
3.1	2.1. Модель требований к ПО	14
3.1.1	2.1.1. Use Case диаграмма	14
3.1.2	2.1.2. Спецификация прецедентов	14
3.1.3	2.1.3. Глоссарий терминов	15
3.2	2.2. Модель предметной области	16
3.2.1	2.2.1. Domain Model (диаграмма классов)	16
3.2.2	2.2.2. Описание сущностей и атрибутов	17
3.2.3	2.2.3. Бизнес-правила	18
3.3	2.3. Архитектурное проектирование	18
3.3.1	2.3.1. Выбор архитектурного стиля	18
3.3.2	2.3.2. Диаграмма пакетов (PCMEF)	19
3.3.3	2.3.3. Описание слоёв и их ответственности	20
3.3.4	2.3.4. Архитектурные решения (ADR)	21
3.4	2.4. Проектирование базы данных	24

3.4.1	2.4.1. ER-диаграмма	24
3.4.2	2.4.2. Физическая модель данных	24
3.4.3	2.4.3. DDL-скрипты	24
3.5	2.5. Детальное проектирование	26
3.5.1	2.5.1. Диаграммы последовательности	26
3.5.2	2.5.2. Диаграммы классов проектирования	29
3.5.3	2.5.3. Применение паттернов GoF	34
4	3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ	36
4.1	3.1. Реализация бизнес-логики	36
4.1.1	3.1.1. Структура проекта	36
4.1.2	3.1.2. Классы-сущности	36
4.1.3	3.1.3. Слой доступа к данным	37
4.1.4	3.1.4. Слой управления	37
4.2	3.2. Рефакторинг и оптимизация	38
4.2.1	3.2.1. Статический анализ кода	38
4.2.2	3.2.2. Применение паттернов (Data Mapper, Identity Map) .	38
4.2.3	3.2.3. Оптимизация запросов	38
4.3	3.3. Пользовательский интерфейс	38
4.3.1	3.3.1. Desktop приложение	38
4.3.2	3.3.2. Web приложение	39
4.3.3	3.3.3. Сравнение реализаций	42
4.4	3.4. Безопасность и транзакции	42
4.4.1	3.4.1. Аутентификация и авторизация	42
4.4.2	3.4.2. Управление транзакциями	43
4.4.3	3.4.3. Защита от атак	43
4.5	3.5. REST API	44
4.5.1	3.5.1. Спецификация OpenAPI	44

4.5.2	3.5.2. Реализация контроллеров	44
4.5.3	3.5.3. Тестирование API	44
5	4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА	45
5.1	4.1. Модульное тестирование	45
5.2	4.2. Интеграционное тестирование	45
5.3	4.3. Системное тестирование	45
5.4	4.4. Нагрузочное тестирование	46
5.5	4.5. Результаты тестирования	46
6	5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ	48
6.1	5.1. Контейнеризация (Docker)	48
6.2	5.2. Инструкция по установке	48
6.3	5.3. Инструкция по настройке	48
6.4	5.4. Требования к окружению	49
7	6. УПРАВЛЕНИЕ ПРОЕКТОМ	50
7.1	6.1. WBS (иерархическая структура работ)	50
7.2	6.2. Диаграмма Ганта	50
7.3	6.3. Оценка трудозатрат (COCOMO)	51
7.4	6.4. Управление рисками	51
8	ЗАКЛЮЧЕНИЕ	53
8.1	Выводы	53
8.2	Результаты	53
8.3	Перспективы развития	54
	Список использованных источников	55
	Приложение А Приложение А. Листинги кода	56
	Приложение Б Приложение Б. Скриншоты интерфейсов	57
	Приложение В Приложение В. Результаты тестирования	58

1 ВВЕДЕНИЕ

1.1 Актуальность

В современном мире мессенджеры стали неотъемлемой частью повседневной жизни. По данным Statista, в 2025 году число пользователей мобильных мессенджеров превысило 3,5 миллиарда человек, а к 2029 году прогнозируется рост до 4,5 миллиарда. В условиях такой высокой конкуренции разработчики стремятся расширять функциональность своих продуктов, добавляя социальные элементы: публикации, комментарии, профили пользователей, сообщества.

Протокол Matrix представляет собой открытый стандарт для децентрализованной коммуникации. Его ключевое преимущество — федеративная архитектура, позволяющая пользователям разных серверов взаимодействовать друг с другом. Однако базовая реализация Matrix-сервера (Synapse, Dendrite) не предоставляет расширенных социальных функций, таких как управление персональной информацией, публикации, комментарии и алиасы в группах.

Существующие социальные сети (Facebook, VK, Telegram) являются централизованными платформами, что порождает ряд проблем: монополизацию данных пользователей, цензуру контента, отсутствие прозрачности в работе алгоритмов. Децентрализованные альтернативы (Mastodon, Lemmy) решают часть этих проблем, но имеют ограниченную интеграцию с мессенджерами и сложность внедрения для корпоративных пользователей.

Таким образом, разработка системы управления профилем пользователя и групповыми чатами на базе протокола Matrix является актуальной задачей, поскольку позволяет:

- расширить функциональность существующих Matrix-инфраструктур без нарушения принципов децентрализации;
- обеспечить пользователям контроль над персональными данными;
- создать единую экосистему коммуникации и социального взаимодействия.

1.2 Цель и задачи

Цель курсового проекта: разработать программную систему управления профилем пользователя и групповыми чатами для мессенджера на базе протокола Matrix с использованием современных архитектурных паттернов и технологий.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области, бизнес-процессов и существующих аналогов.
2. Спроектировать модель требований к программному обеспечению (Use Case, спецификация прецедентов, глоссарий).
3. Разработать модель предметной области (Domain Model, бизнес-правила).
4. Выполнить архитектурное проектирование на основе паттерна PCMEF с адаптацией под feature-first и гексагональные принципы.
5. Спроектировать реляционную базу данных (ER-диаграмма, физическая модель, DDL-скрипты).
6. Выполнить детальное проектирование (диаграммы последовательности, классов проектирования, паттерны GoF).
7. Реализовать бизнес-логику на Java Spring Boot с соблюдением слоистой архитектуры.
8. Разработать пользовательский интерфейс на React с интеграцией через REST API.
9. Обеспечить безопасность (аутентификация через Matrix OpenID, JWT-авторизация, управление транзакциями).
10. Разработать и документировать REST API (OpenAPI 3.1).
11. Выполнить модульное, интеграционное и системное тестирование.
12. Подготовить инструкции по развёртыванию (Docker, Kubernetes, Helm).
13. Оценить трудозатраты и спланировать управление проектом.

1.3 Объект и предмет исследования

Объект исследования — процесс разработки программного обеспечения для расширения функциональности мессенджеров на базе протокола Matrix.

Предмет исследования — методы и средства проектирования и реализации системы управления профилем пользователя и групповыми чатами в контексте enterprise-архитектуры с применением паттерна PCMEF, гексагональных принципов и современных фреймворков (Spring Boot, React).

2 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

2.1 1.1. Описание предметной области

Предметная область охватывает функциональность социальных сетей и мессенджеров, интегрированных с протоколом Matrix. Ключевые понятия:

- **Профиль пользователя** — совокупность персональных данных (имя, аватар, статус, дата рождения, адрес, любимый трек), доступных для просмотра и редактирования.
- **Публикация (пост)** — сообщение, содержащее текст и/или медиаконтент (изображение, видео, аудио, ссылка), размещаемое в ленте профиля.
- **Комментарий** — ответ к публикации, поддерживающий древовидную структуру.
- **Групповой чат** — многопользовательское пространство для общения.
- **Алиас (роль)** — отображаемое имя пользователя в контексте конкретной группы.
- **Сообщество** — публичное или приватное объединение пользователей с общими интересами.

2.2 1.2. Анализ бизнес-процессов (IDEF0)

Бизнес-процессы системы описываются с помощью методологии IDEF0. На верхнем уровне (A-0) система получает на вход:

- запросы пользователей (создание поста, редактирование профиля);
- данные аутентификации (Matrix OpenID токен);
- медиаконтент (файлы изображений, видео, аудио).

На выходе система производит:

- обновлённые профили и публикации;
- уведомления подписчикам;
- статистику активности.

Управление осуществляется через политики безопасности и бизнес-правила. Механизмами выступают Spring Boot приложение, PostgreSQL база данных, React frontend.

Business Use Case — msocial (Matrix Social Network)

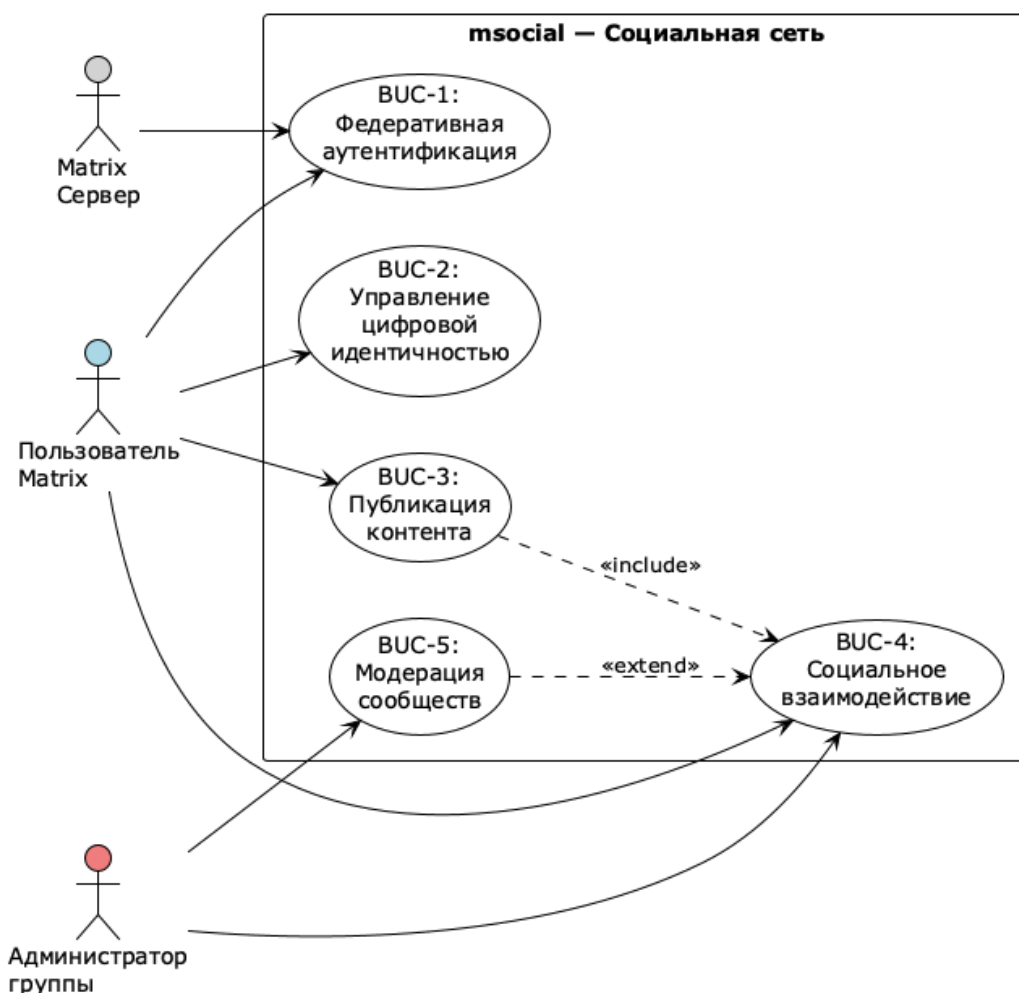


Рисунок 1 — Диаграмма бизнес-use case (BUC)

2.3 1.3. SWOT-анализ текущего состояния

Таблица 1 — SWOT-анализ системы на базе протокола Matrix

Сильные стороны (Strengths)	Слабые стороны (Weaknesses)
Открытый протокол Matrix; децентрализованная архитектура; контроль пользователя над данными.	Недостаточная зрелость экосистемы по сравнению с коммерческими аналогами; сложность настройки для конечных пользователей.

Возможности (Opportunities)	Угрозы (Threats)
Рост спроса на приватность; корпоративный сектор ищет альтернативы Slack/Teams; регуляторное давление на Big Tech.	Конкуренция с хорошо финансируемыми платформами; фрагментация федеративной сети; проблемы масштабирования.

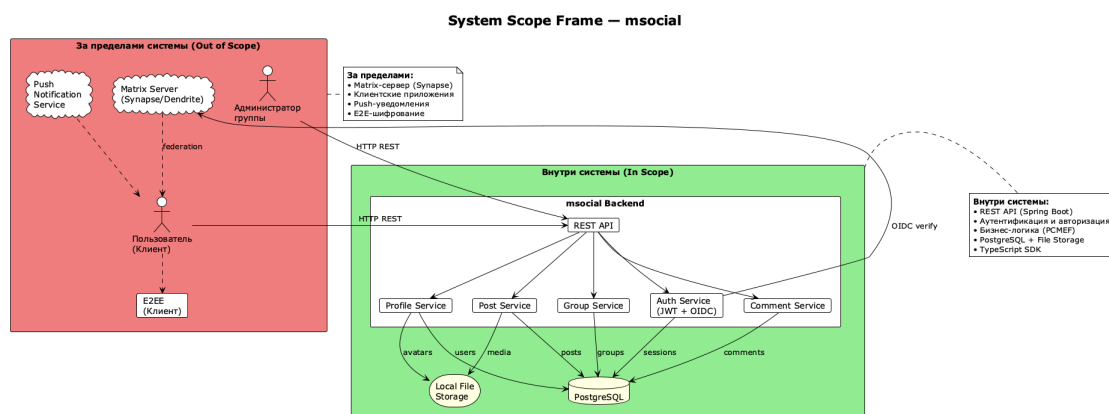


Рисунок 2 — Стратегическая рамка системы

2.4 1.4. Анализ аналогов и конкурентов

Таблица 2 — Сравнительный анализ аналогов и конкурентов

Продукт	Архитектура	Социальные функции	Открытость
Telegram	Централизованная	Каналы, комментарии, боты	Частично (клиенты открыты)
Discord	Централизованная	Серверы, роли, каналы	Нет
Mastodon	Федеративная (ActivityPub)	Микроблоги, ленты	Да
Element (Matrix)	Федеративная (Matrix)	Чаты, комнаты	Да
Предлагаемая система	Федеративная (Matrix)	Профили, посты, комментарии, алиасы, сообщества	Да

Ключевое конкурентное преимущество разрабатываемой системы — глубокая интеграция социальных функций (профили, публикации, комментарии) непосредственно в экосистему Matrix, что позволяет использовать существующую федеративную инфраструктуру без необходимости создания отдельной платформы.

2.5 1.5. Обоснование необходимости разработки

Разработка системы обосновывается следующими факторами:

1. **Отсутствие комплексного решения** — ни один из существующих продуктов на базе Matrix не предоставляет полноценных социальных функций (публикации, комментарии, персональные профили) в единой экосистеме.
2. **Корпоративный спрос** — компании, использующие Matrix для внутренней коммуникации, нуждаются в инструментах для публикации новостей, управления профилями сотрудников и модерирования групп.
3. **Технологическая зрелость** — протокол Matrix достиг версии 1.0, спецификации стабилизированы, SDK доступны для всех популярных платформ.
4. **Архитектурная целесообразность** — применение паттерна PCMEF с гексагональными принципами обеспечивает масштабируемость, тестируемость и возможность эволюции системы от монолита к микросервисам.

2.6 1.6. Экономическое обоснование (ROI)

Экономическая эффективность проекта оценивается через снижение затрат на лицензирование коммерческих аналогов и повышение продуктивности пользователей.

Таблица 3 — Экономическое обоснование разработки системы

Показатель	Значение	Примечание
Стоимость лицензий Slack Enterprise (100 пользователей/год)	\$15 000	Внешние затраты

Показатель	Значение	Примечание
Стоимость разработки и поддержки системы (год)	\$8 000	Внутренние затраты
Экономия в первый год	\$7 000	Разница
ROI (год 1)	87,5%	(Экономия / Затраты) * 100%
Срок окупаемости	13,7 месяцев	При линейной амортизации

3 2. ПРОЕКТНАЯ ЧАСТЬ

3.1 2.1. Модель требований к ПО

3.1.1 2.1.1. Use Case диаграмма

Диаграмма вариантов использования определяет три актора и десять прецедентов:

- **Акторы:** Пользователь, Администратор группы, Система.
- **Прецеденты:** управление профилем, управление публикациями, комментирование, управление персональной информацией, история аватаров, управление группами, назначение алиасов, изменение имени участника, тег сообщества, список сообществ.

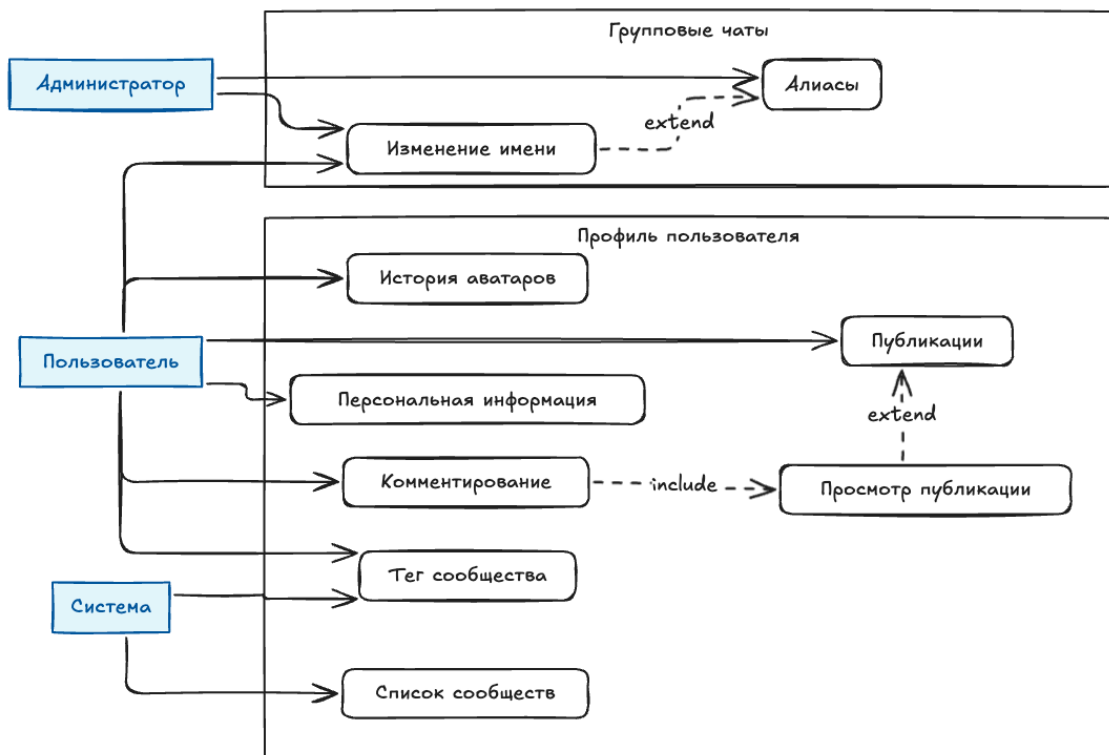


Рисунок 3 — Диаграмма Use Case

3.1.2 2.1.2. Спецификация прецедентов

Ключевые прецеденты детализированы в соответствии с шаблоном RUP:

UC3: Управление публикациями

- Предусловия: пользователь авторизован, имеет права на публикацию, контент в пределах лимитов.
- Постусловия: публикация создана, отображается в ленте, уведомления отправлены подписчикам.
- Основной поток: создание → валидация → сохранение → отображение.
- Исключения: ошибка загрузки медиа, нарушение правил контента, превышение дневного лимита.

UC4: Управление персональной информацией

- Предусловия: пользователь авторизован, профиль доступен для редактирования.
- Постусловия: данные сохранены, профиль обновлён для всех пользователей.
- Основной поток: открытие формы → загрузка текущих значений → редактирование → валидация → сохранение.
- Исключения: ошибка валидации (некорректный формат), ошибка сервера, доступ запрещён.

3.1.3 2.1.3. Глоссарий терминов

Таблица 4 — Глоссарий терминов предметной области

Термин	Определение
Matrix ID	Уникальный идентификатор пользователя в федеративной сети Matrix (например, @user:example.org).
OpenID Token	Токен аутентификации, выдаваемый Matrix-сервером для верификации личности пользователя.
Alias	Отображаемое имя или роль пользователя в контексте конкретной группы.
Community Tag	Постфикс, автоматически добавляемый к имени пользователя при публикации от имени сообщества.
Soft Delete	Маркировка записи как удалённой без физического удаления из базы данных.
JWT	JSON Web Token — токен доступа, содержащий claims пользователя и сессии.

Термин	Определение
PCMEF	Presentation-Control-Mediator-Entity-Foundation — паттерн многослойной архитектуры.

3.2 2.2. Модель предметной области

3.2.1 2.2.1. Domain Model (диаграмма классов)

Domain Model построена с использованием многоуровневой архитектуры, характерной для enterprise-приложений на платформе Spring. Модель включает шесть слоёв:

1. Controller Layer — обработка HTTP запросов, валидация, маппинг DTO.
2. Service Layer — бизнес-логика, транзакции, оркестрация репозитория.
3. Repository Layer — доступ к данным, CRUD операции.
4. Entity Layer — модели данных, маппинг на таблицы БД.
5. DTO Layer — передача данных между слоями.
6. Exception Layer — обработка ошибок и HTTP статусов.

Ключевые сущности: User, PersonalInfo, Avatar, Post, Comment, GroupChat, GroupMember, Community.

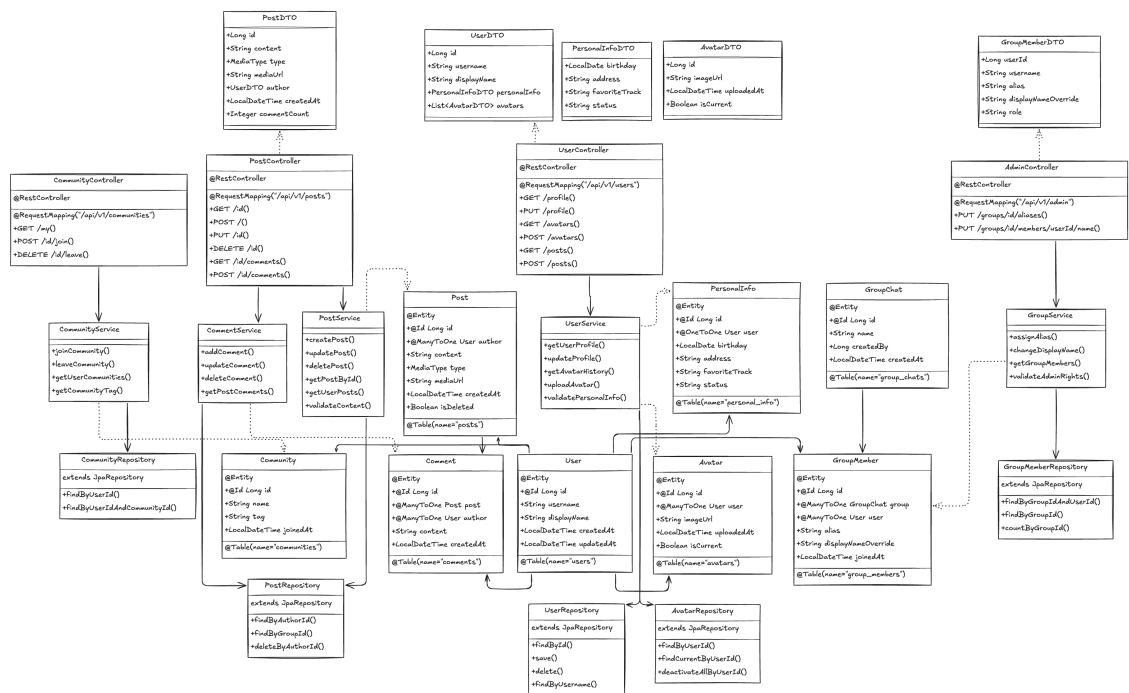


Рисунок 4 – Domain Model – модель предметной области

3.2.2 2.2.2. Описание сущностей и атрибутов

Таблица 5 – Описание слоёв и компонентов Domain Model

Слой	Компоненты	Ответственность	Аннотации Spring
Controller	UserController, PostController, AdminController, CommunityController	Обработка HTTP запросов, валидация, маппинг DTO	@RestController, @RequestMapping, @GetMapping, @PostMapping
Service	UserService, PostService, CommentService, GroupService, CommunityService	Бизнес-логика, транзакции, оркестрация	@Service, @Transactional
Repository	UserRepository, PostRepository, AvatarRepository, GroupMemberRepository, CommunityRepository	Доступ к данным, CRUD	@Repository, extends JpaRepository
Entity	User, PersonalInfo, Avatar, Post, Comment, GroupChat,	Модель данных, ORM-маппинг	@Entity, @Table, @Id, @ManyToOne, @OneToOne

Слой	Компоненты	Ответственность	Аннотации Spring
	GroupMember, Community		
DTO	UserDTO, PostDTO, PersonalInfoDTO, AvatarDTO, GroupMemberDTO	Передача данных, скрывание внутренней структуры	Plain Java Classes
Exception	ValidationException, NotFoundException, AccessDeniedException	Обработка ошибок, уведомление	@ResponseStatus, @ControllerAdvice

3.2.3 2.2.3. Бизнес-правила

Таблица 6 — Бизнес-правила и ограничения предметной области

Правило	Описание	Последствия нарушения
BR-001	Один пользователь = один активный аватар	Конфликт отображения профиля
BR-002	Алиас может назначить только администратор группы	Нарушение прав доступа
BR-003	Личная группа может быть только одна на пользователя	Конфликт приватного пространства
BR-004	Тег сообщества отображается только для активных участников	Некорректная информация о членстве
BR-005	Изменение имени в группе не влияет на глобальное имя	Путаница в идентификации пользователя

3.3 2.3. Архитектурное проектирование

3.3.1 2.3.1. Выбор архитектурного стиля

Архитектурный паттерн PCMEF выбран по следующим причинам:

1. **Чёткое разделение ответственности** — пять строго определённых слоёв исключают дублирование функциональности.
2. **Однонаправленная направленность зависимостей** — каждый слой зависит только от нижележащего, что гарантирует отсутствие циклических связей.

3. **Совместимость с Spring-экосистемой** — слои PCMEF естественным образом отображаются на стандартные аннотации Spring.
4. **Возможность адаптации** — PCMEF допускает модификацию под конкретную предметную область без потери основных принципов.
5. **Проверенная практика** — паттерн широко применяется в enterprise-приложениях.

3.3.2 2.3.2. Диаграмма пакетов (PCMEF)

Система организована по бизнес-фичам (feature-first), внутри каждой из которых присутствуют собственные PCMEF-слои:

```
com.app.backendapi
├─ feature/
│   ├─ user/      (dto, controller, service, entity, repository)
│   ├─ post/      (аналогично)
│   ├─ comment/   (аналогично)
│   ├─ group/     (аналогично)
│   └─ community/ (аналогично)
├─ common/
│   ├─ exceptions/ (BaseException, ValidationException,
│   │               NotFoundException)
│   ├─ security/   (AuthFilter, JwtProvider, SecurityConfig)
│   ├─ config/     (WebMvcConfig, SwaggerConfig, DBConfig)
│   └─ utils/      (DateUtil, StringUtils, ValidatorUtil)
```

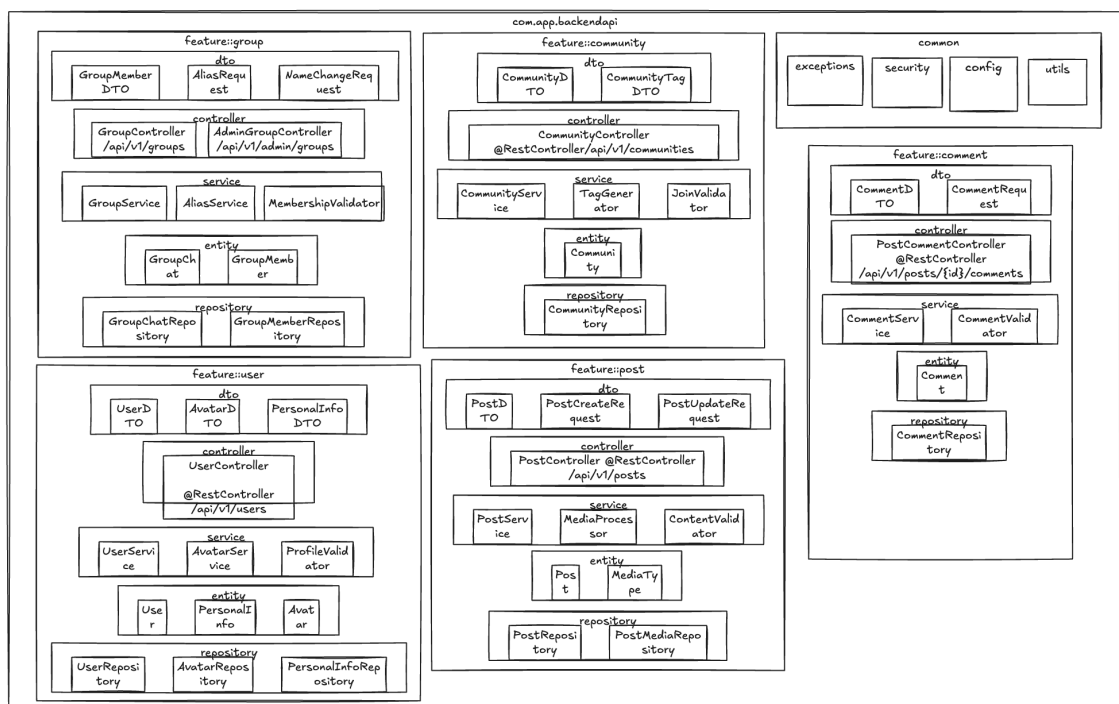


Рисунок 5 — Диаграмма пакетов PCMEF backend-слоя

3.3.3 2.3.3. Описание слоёв и их ответственности

Таблица 7 — Распределение компонентов по слоям PCMEF

Слой PCMEF	Компоненты проекта	Назначение
Presentation	UserDTO, PostDTO, CommentDTO, PostCreateRequest, PostUpdateRequest, CommentRequest	Форматы данных API, маппинг запросов/ответов
Control	UserController, PostController, PostCommentController, GroupController, AdminGroupController, CommunityController	Обработка HTTP-запросов, валидация, маршрутизация
Mediator	UserService, PostService, CommentService, GroupService, CommunityService, AvatarService, MediaProcessor,	Бизнес-логика, транзакции, оркестрация

Слой PCMEF	Компоненты проекта	Назначение
	TagGenerator, ProfileValidator, ContentValidator	
Entity	User, Post, Comment, GroupChat, GroupMember, Community, PersonalInfo, Avatar, MediaType	Модели предметной области, ORM-маппинг
Foundation	UserRepository, PostRepository, PostMediaRepository, CommentRepository, GroupChatRepository, GroupMemberRepository, CommunityRepository, AvatarRepository, PersonalInfoRepository	Доступ к данным, CRUD-операции

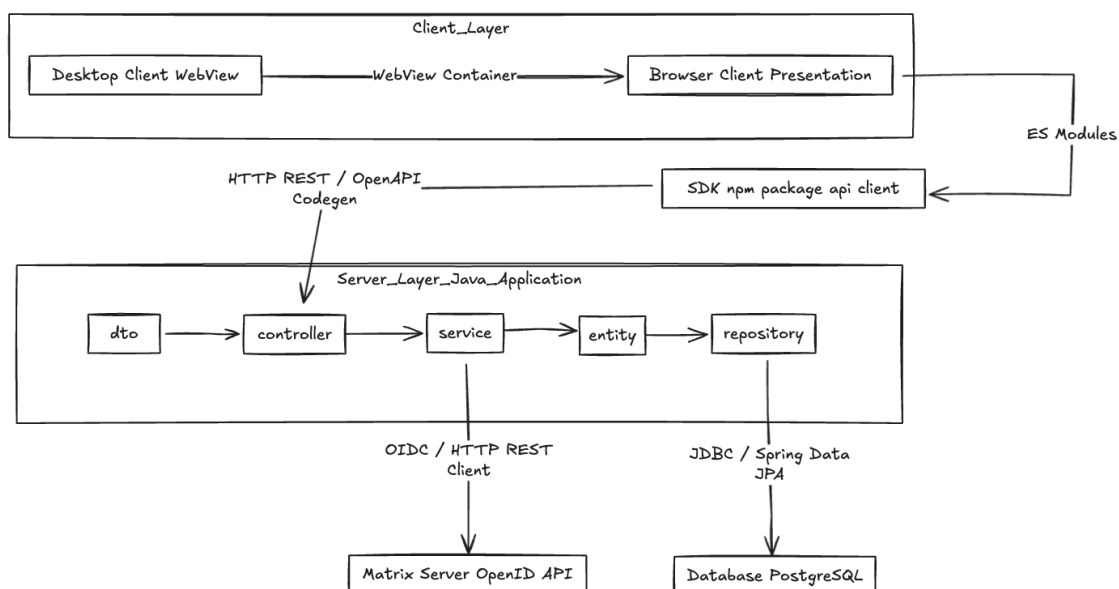


Рисунок 6 — Диаграмма инфраструктуры и развёртывания

3.3.4 2.3.4. Архитектурные решения (ADR)

ADR-001: Выбор архитектурного паттерна PCMEF

- Контекст: необходимо обеспечить чёткое разделение ответственности и масштабируемость.

- Решение: использовать PCMEF с адаптацией под feature-first.
- Последствия: упрощение навигации по коду, независимое тестирование фич, возможность выделения микросервисов.

ADR-002: Выбор базы данных и ORM

- Контекст: требуется надёжное хранение реляционных данных с поддержкой транзакций.
- Решение: PostgreSQL + Spring Data JPA.
- Последствия: стандартизация доступа к данным, миграции через Flyway.

ADR-003: Стратегия аутентификации

- Контекст: интеграция с существующей Matrix-инфраструктурой.
- Решение: Matrix OpenID Federation + внутренние JWT (access/refresh).
- Последствия: единый вход для Matrix-пользователей, stateless API, ротация токенов.

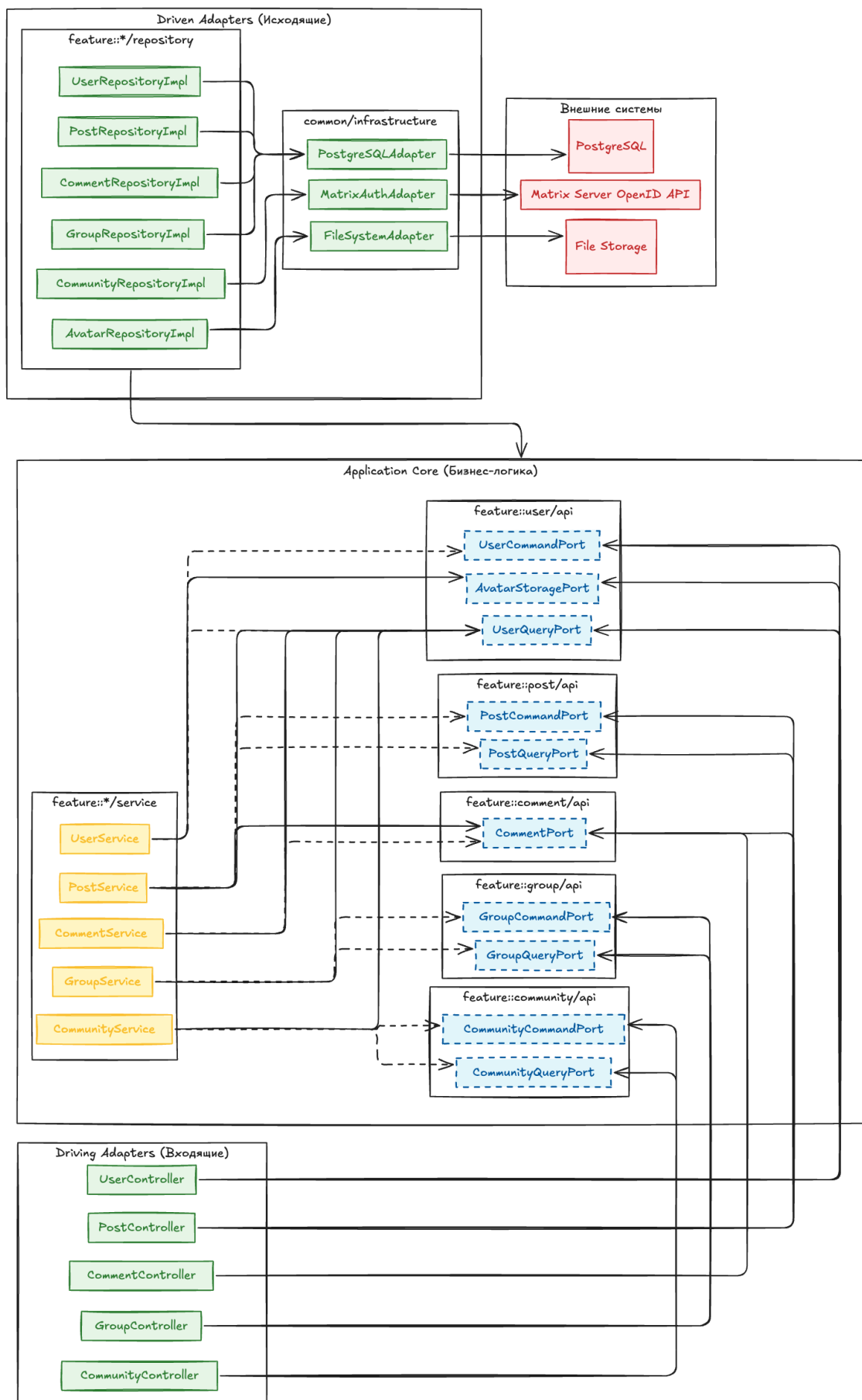


Рисунок 7 – Диаграмма интерфейсов (Ports & Adapters)

3.4 2.4. Проектирование базы данных

3.4.1 2.4.1. ER-диаграмма

Логическая модель данных включает восемь сущностей:

- `users` — центральная сущность, 1:1 с `personal_infos`, 1:N с `avatars`, `posts`, `comments`, `personal_groups`.
- `personal_infos` — личная информация пользователя (дата рождения, адрес, статус, любимый трек).
- `avatars` — история загрузок аватаров с флагом `is_current`.
- `communities` — сообщества с уникальным тегом.
- `group_members` — ассоциативная сущность M:N между `communities` и `users`, содержит алиас и переопределение имени.
- `posts` — публикации с типом медиа и ссылкой на контент.
- `comments` — комментарии к постам, древовидная структура через `parent_id`.
- `personal_groups` — личные группы пользователей.

3.4.2 2.4.2. Физическая модель данных

Нормализация выполнена до третьей нормальной формы (3НФ):

- 1НФ: все атрибуты атомарны, типы приведены к SQL-типам.
- 2НФ: все сущности имеют суррогатный ключ `id`, частичные зависимости отсутствуют.
- 3НФ: добавлены `UNIQUE` ограничения для гарантии 1:1 связи (`personal_infos.user_id`), частичный индекс для активного аватара, уникальный композитный ключ (`group_members.group_id`, `group_members.user_id`).

3.4.3 2.4.3. DDL-скрипты

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  display_name VARCHAR(100) NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
```



```

        last_active TIMESTAMP WITH TIME ZONE
    );

CREATE TABLE personal_infos (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL UNIQUE,
    birthday DATE,
    address VARCHAR(255),
    favorite_track_url VARCHAR(512),
    status VARCHAR(255),
    CONSTRAINT fk_personal_info_user FOREIGN KEY (user_id)
        REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE avatars (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    image_url VARCHAR(512) NOT NULL,
    uploaded_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    is_current BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_avatar_user FOREIGN KEY (user_id)
        REFERENCES users(id) ON DELETE CASCADE
);

CREATE UNIQUE INDEX idx_current_avatar ON avatars(user_id) WHERE
is_current = TRUE;

CREATE TABLE communities (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    tag VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE group_members (
    id SERIAL PRIMARY KEY,
    group_id INTEGER NOT NULL,
    user_id INTEGER NOT NULL,
    alias VARCHAR(100),
    display_name_override VARCHAR(100),
    CONSTRAINT fk_member_group FOREIGN KEY (group_id)
        REFERENCES communities(id) ON DELETE CASCADE,
    CONSTRAINT fk_member_user FOREIGN KEY (user_id)
        REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT uq_group_member UNIQUE(group_id, user_id)
);

```

```

);

CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    author_id INTEGER NOT NULL,
    content TEXT,
    media_type VARCHAR(20) CHECK (media_type IN ('text', 'image',
'video', 'audio', 'link')),
    media_url VARCHAR(512),
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_post_author FOREIGN KEY (author_id)
        REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE comments (
    id SERIAL PRIMARY KEY,
    post_id INTEGER NOT NULL,
    author_id INTEGER NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_comment_post FOREIGN KEY (post_id)
        REFERENCES posts(id) ON DELETE CASCADE,
    CONSTRAINT fk_comment_author FOREIGN KEY (author_id)
        REFERENCES users(id) ON DELETE CASCADE
);

```

3.5 2.5. Детальное проектирование

3.5.1 2.5.1. Диаграммы последовательности

Для детального проектирования выбраны три ключевых сценария:

1. **UC4: Управление персональной информацией** — редактирование профиля с валидацией и частичным обновлением.
2. **UC3: Создание публикации** — создание поста с валидацией контента и сохранением в БД.
3. **UC3-2a: Загрузка и валидация медиафайлов** — циклическая обработка файлов с ветвлением по результатам валидации.

Взаимодействие объектов следует направлению PCMEF: Presentation → Controller → Mediator → Foundation → Entity.

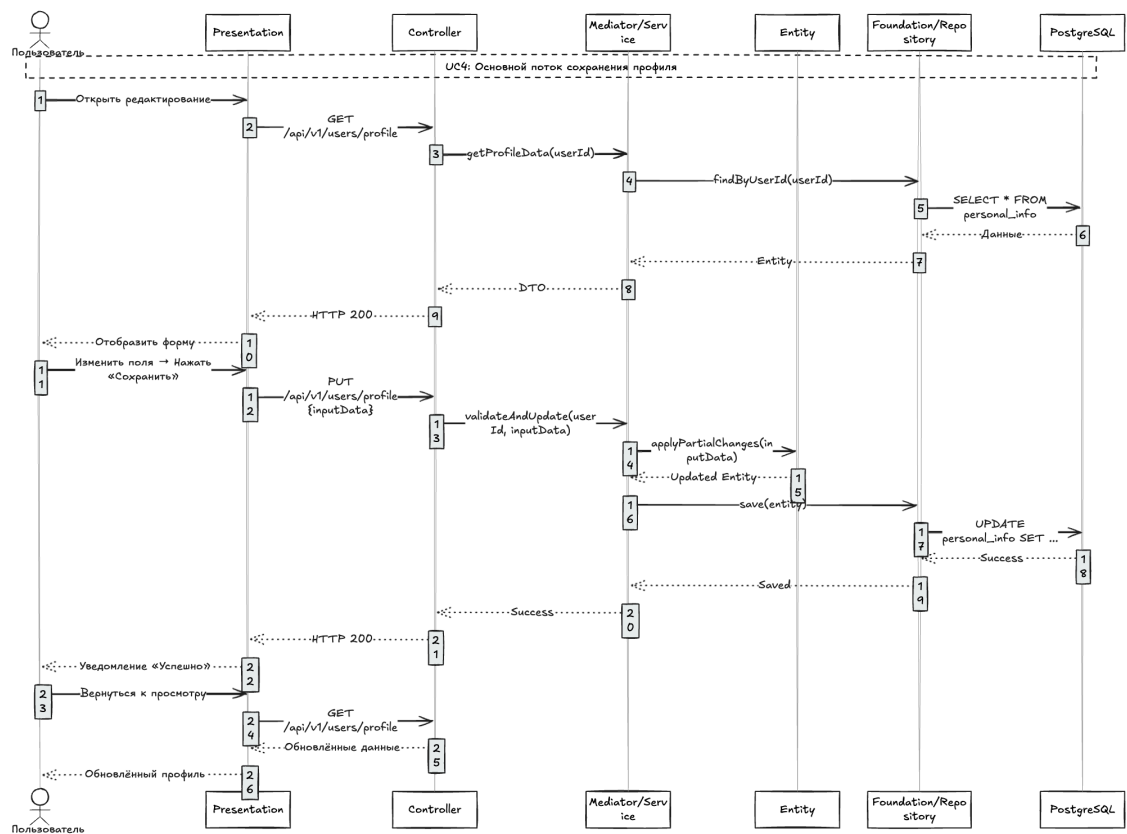


Рисунок 8 — Диаграмма последовательности UC4: основной поток сохранения профиля

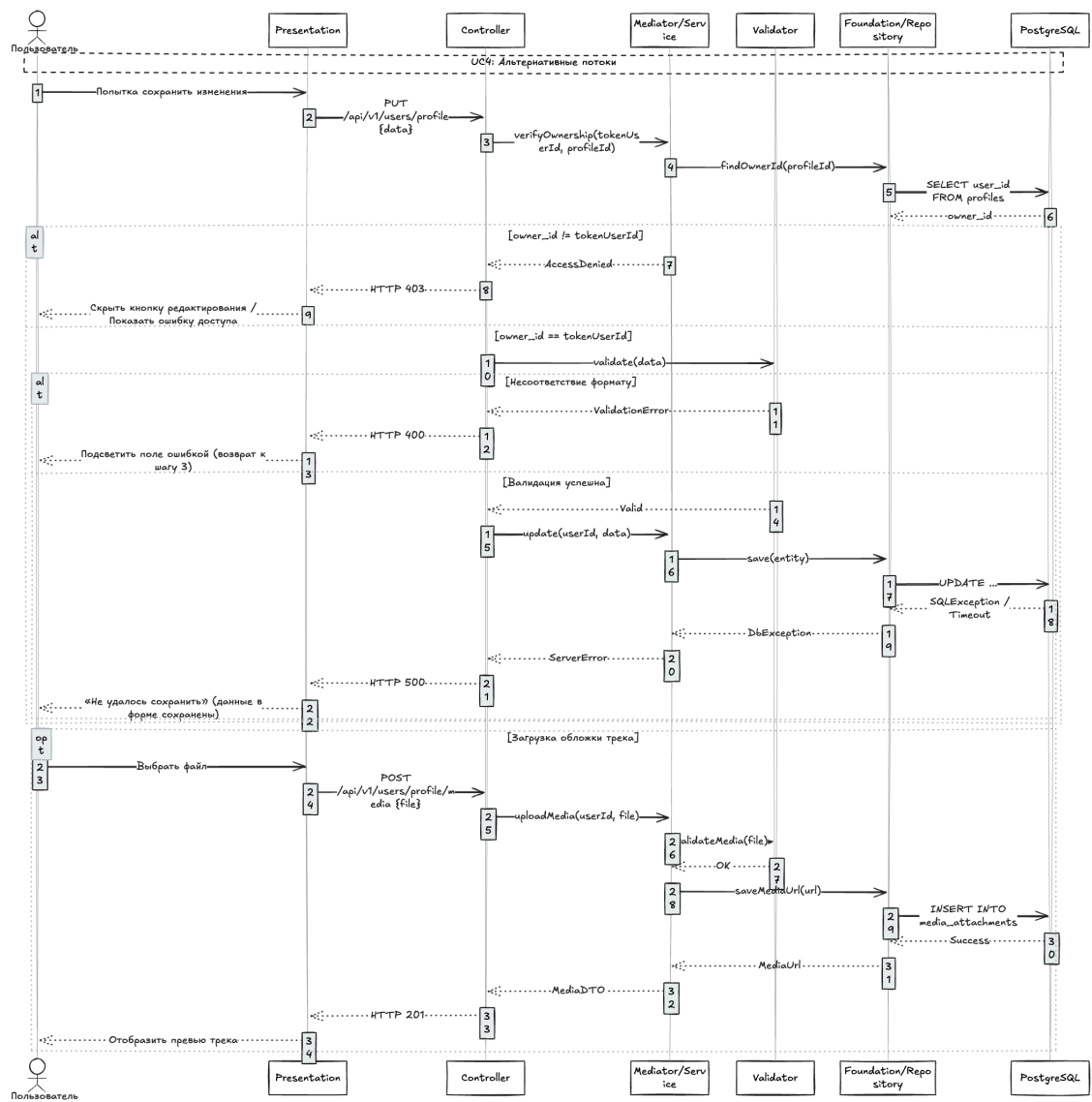


Рисунок 9 — Диаграмма последовательности UC4: альтернативные потоки

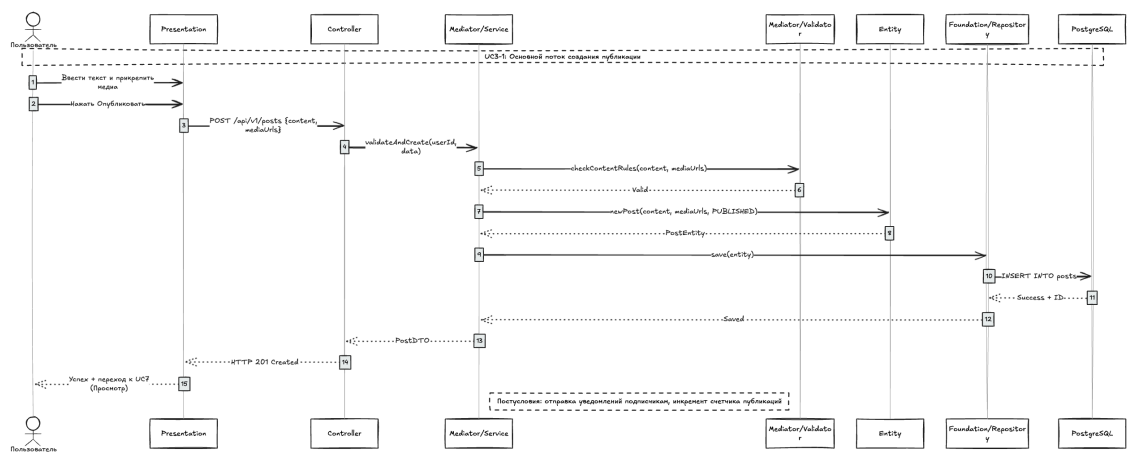


Рисунок 10 — Диаграмма последовательности UC3-1: создание публикации

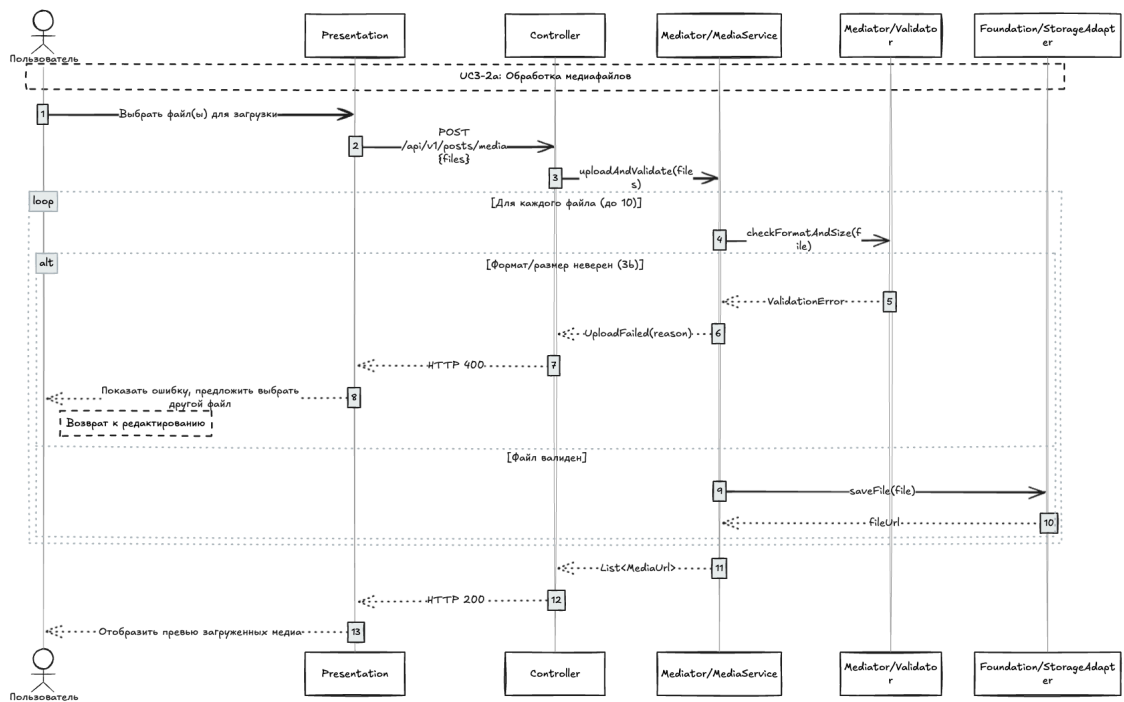


Рисунок 11 — Диаграмма последовательности UC3-2a: обработка медиафайлов

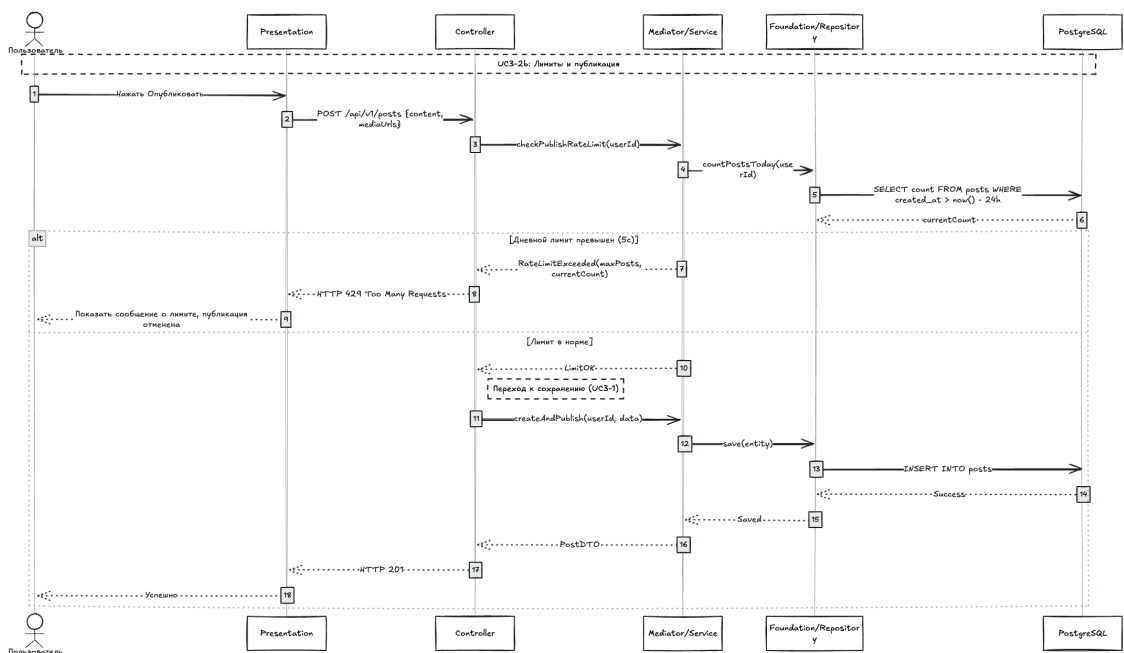


Рисунок 12 — Диаграмма последовательности UC3-2b: проверка лимитов

3.5.2 2.5.2. Диаграммы классов проектирования

Уточнённая диаграмма классов разделена на доменные модули (feature-first):

- feature::user — порты UserQueryPort, UserCommandPort, AvatarStoragePort; сервисы UserService, AvatarService; валидатор ProfileValidator.
- feature::auth — порты AuthCommandPort, TokenValidationPort, OidcVerificationPort, TokenGenerationPort, SessionManagementPort; адаптер MatrixFederationAdapter.
- feature::post — порты PostQueryPort, PostCommandPort, MediaProcessorPort; валидатор ContentValidator.
- feature::comment — порты CommentQueryPort, CommentCommandPort, PostContextPort; валидатор CommentValidator.

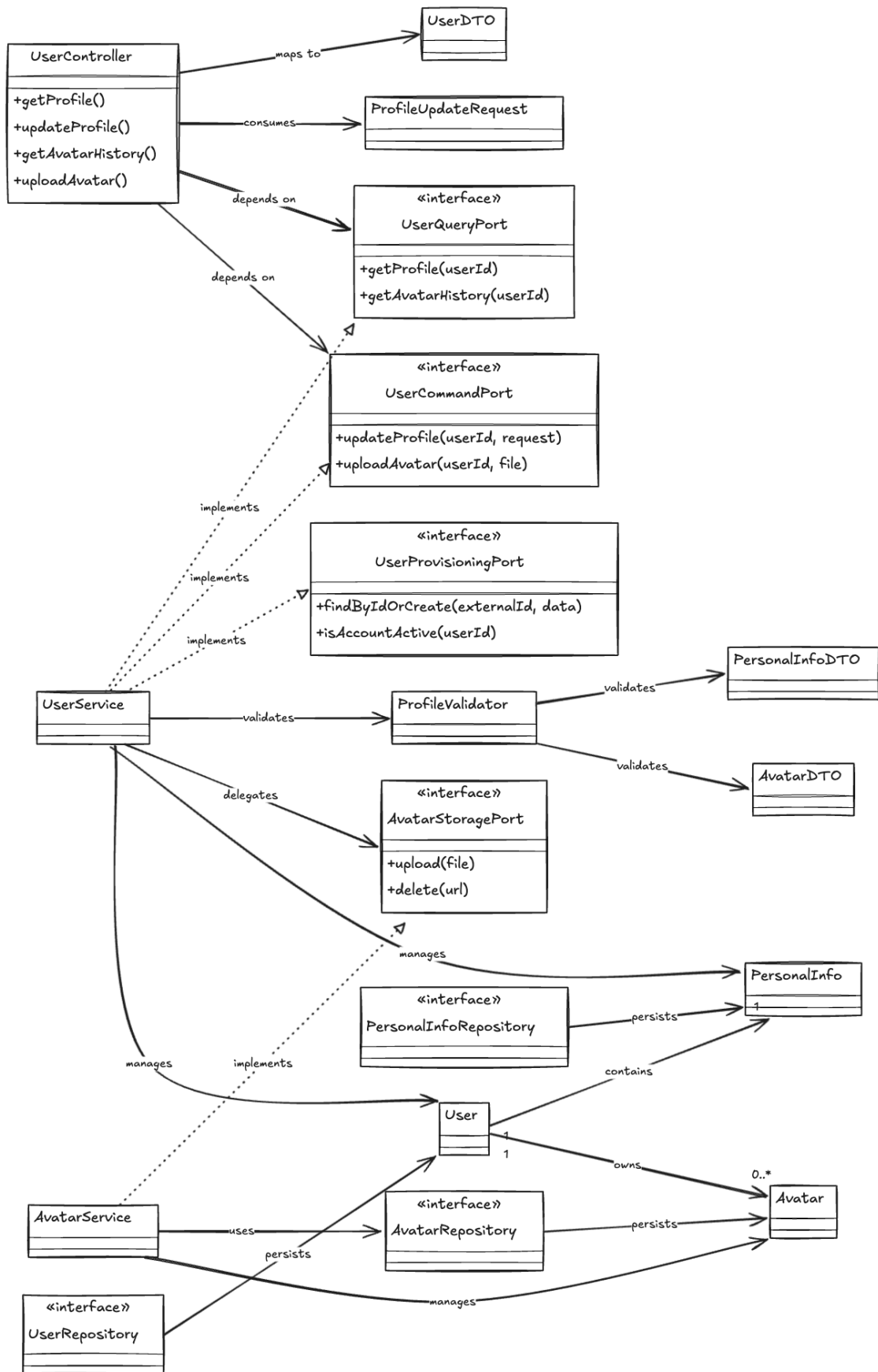


Рисунок 13 – Диаграмма классов: управление пользователями

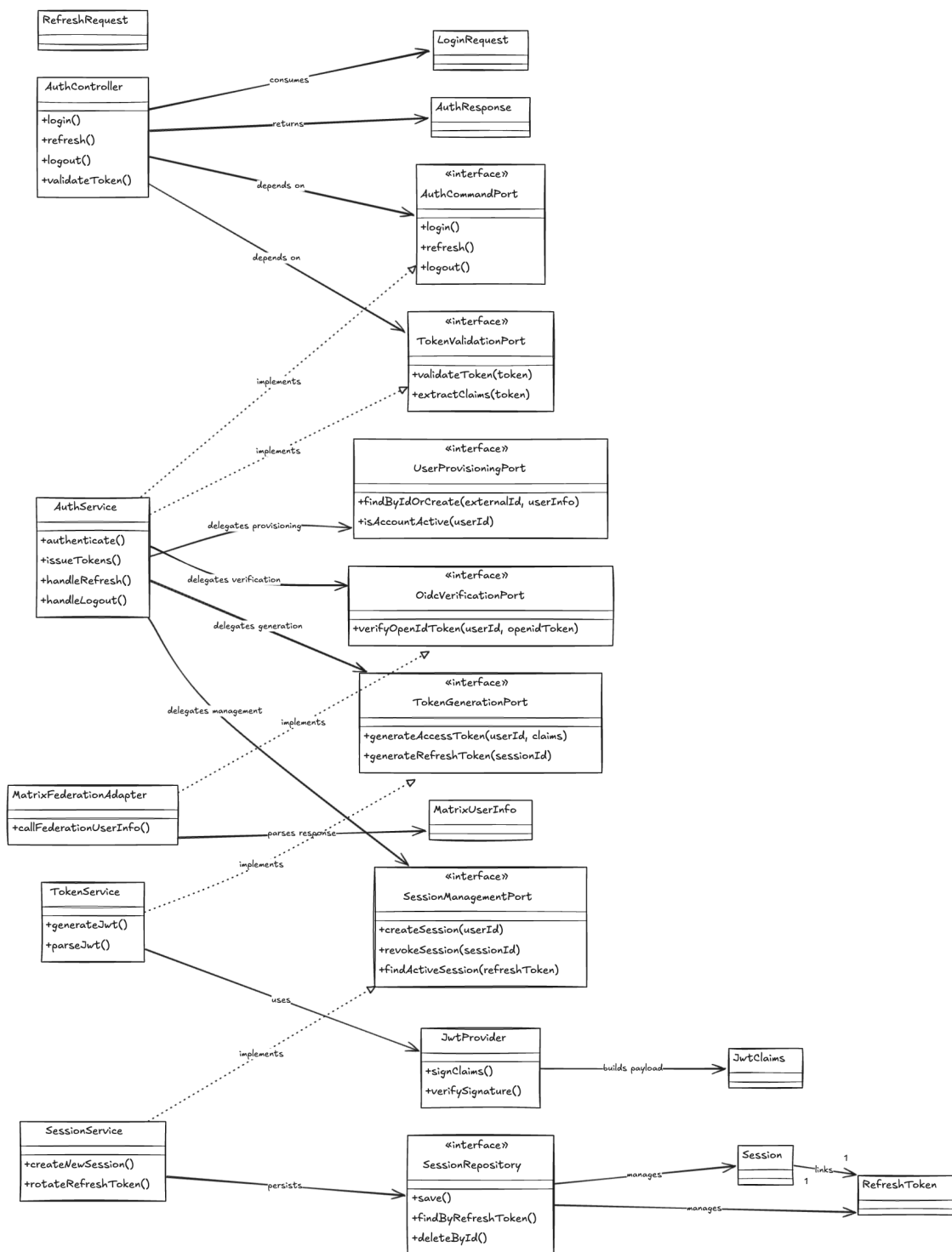


Рисунок 14 — Диаграмма классов: аутентификация

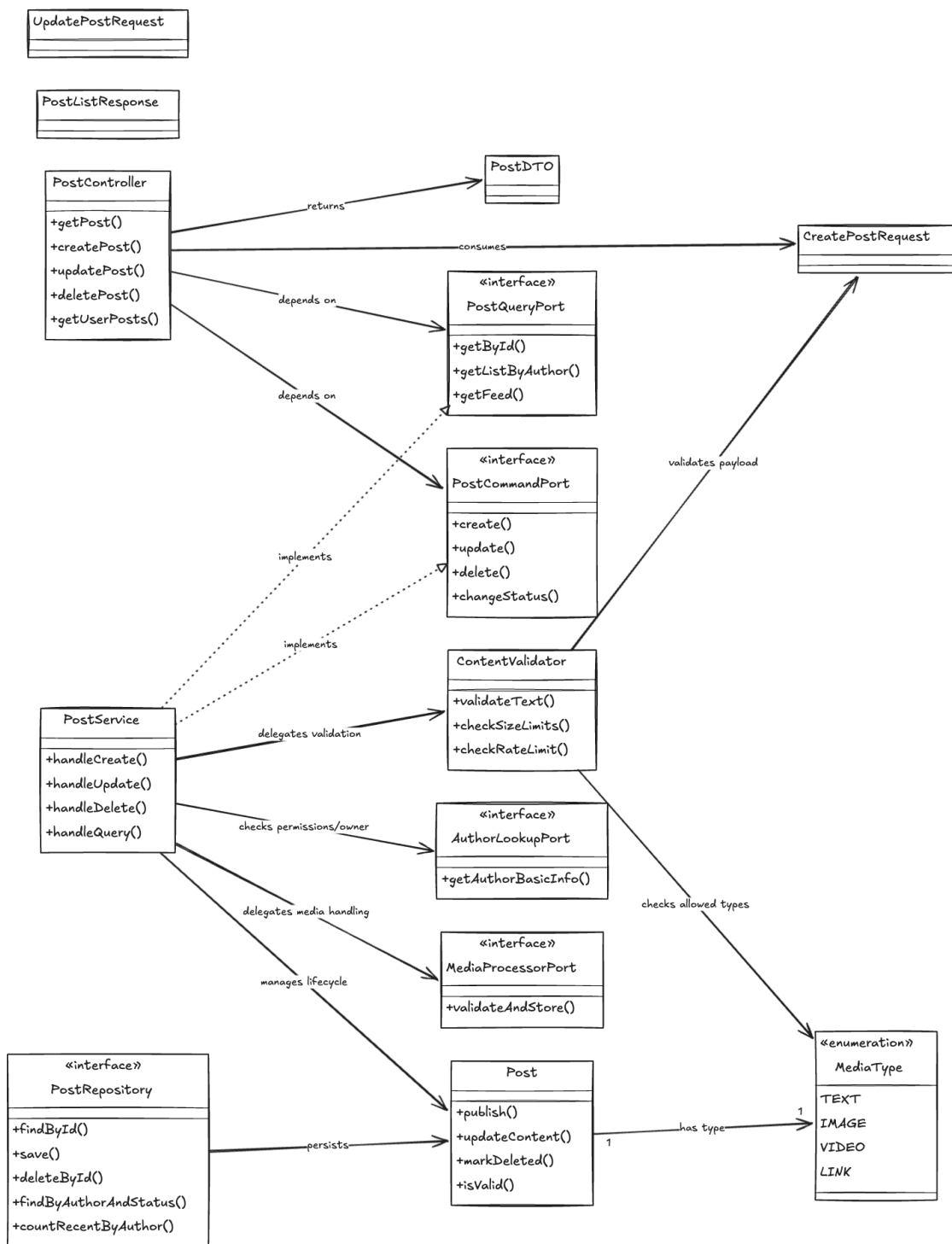


Рисунок 15 — Диаграмма классов: публикации

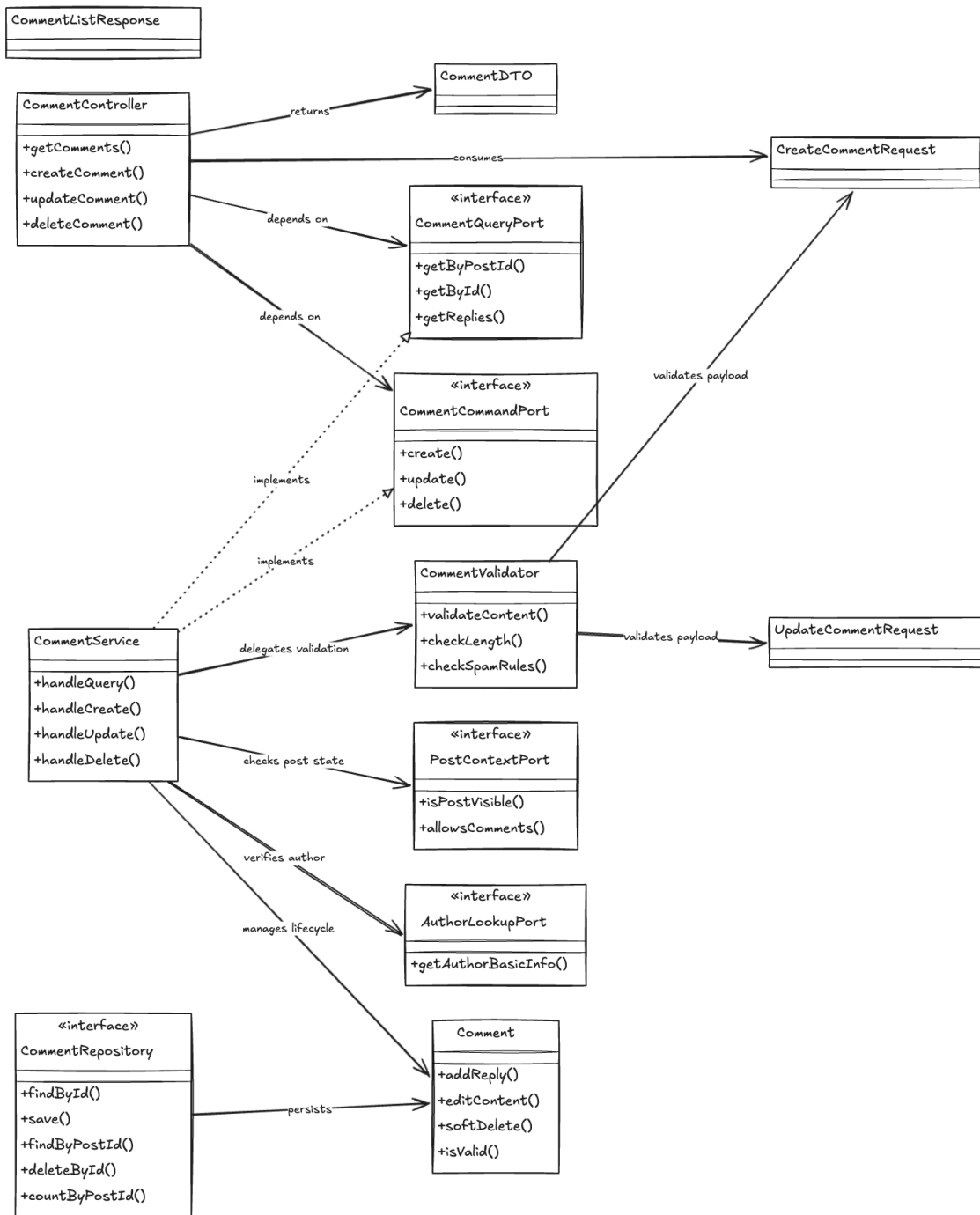


Рисунок 16 — Диаграмма классов: комментарии

3.5.3 2.5.3. Применение паттернов GoF

Decorator — кеширование, rate-limiting и метрики для портов через декорирование интерфейсов (CachedUserQueryPort).

Chain of Responsibility — валидация комментариев и постов через цепочку обработчиков (LengthValidator → SpamValidator → ToxicityValidator).

Adapter — интеграция внешних систем (Matrix Federation, S3) через адаптеры (MatrixFederationAdapter, MediaStorageAdapter).

Builder — конструирование DTO и запросов с множеством опциональных полей через Lombok @Builder.

4 3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

4.1 3.1. Реализация бизнес-логики

4.1.1 3.1.1. Структура проекта

```
msocial/
├─ src/main/java/lghdnov/msocial/
│   └─ feature/
│       └─ user/          (api, controller, presentation, service,
entity, repository, infrastructure)
│           └─ auth/      (аналогично)
│           └─ post/      (аналогично)
│           └─ comment/   (аналогично)
│           └─ group/     (аналогично)
│           └─ community/ (аналогично)
│       └─ common/
│           └─ exceptions/ (ValidationException, NotFoundException,
AccessDeniedException)
│           └─ security/   (AuthFilter, JwtProvider, SecurityConfig)
│           └─ config/     (WebMvcConfig, SwaggerConfig, DBConfig)
│           └─ utils/      (DateUtil, StringUtils)
├─ src/main/resources/
│   └─ application.yml
│   └─ db/migration/      (Flyway миграции)
└─ src/test/
    └─ java/              (Unit и интеграционные тесты)
    └─ resources/
```

4.1.2 3.1.2. Классы-сущности

Сущность `Session` инкапсулирует бизнес-правило активности сессии:

```
@Entity
@Table(name = "auth_sessions")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Session {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private
Long id;
    @Column(name = "user_id", nullable = false) private Long userId;
    @Column(name = "refresh_token", unique = true, length = 512)
private String refreshToken;
```

```

        @Column(name = "refresh_token_expires_at", nullable = false)
private Instant refreshTokenExpiresAt;
        @Column(name = "created_at", nullable = false) private Instant
createdAt;
        @Column(name = "revoked_at") private Instant revokedAt;
        @Version private Long version;

        public boolean isActive() {
            return revokedAt == null &&
refreshTokenExpiresAt.isAfter(Instant.now());
        }
    }
}

```

4.1.3 3.1.3. Слой доступа к данным

Репозитории реализованы на Spring Data JPA с кастомными методами:

```

@Repository
public interface SessionRepository extends JpaRepository<Session,
Long> {
    Optional<Session> findByRefreshToken(String refreshToken);
    void deleteByRefreshToken(String refreshToken);
}

```

4.1.4 3.1.4. Слой управления

Контроллеры используют порты (интерфейсы) вместо конкретных сервисов, обеспечивая инверсию зависимостей:

```

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {
    private final AuthCommandPort authCommandPort;
    private final TokenValidationPort tokenValidationPort;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody
LoginRequest request) {
        return ResponseEntity.ok(authCommandPort.login(request));
    }
    // ...
}

```

4.2 3.2. Рефакторинг и оптимизация

4.2.1 3.2.1. Статический анализ кода

Проект настроен на выполнение статического анализа через Gradle плагины (SpotBugs, PMD). Основные метрики:

- Покрытие кода тестами (JaCoCo): > 60% для критичных модулей.
- Максимальная цикломатическая сложность методов: ≤ 10.
- Отсутствие критических уязвимостей зависимостей (OWASP Dependency Check).

4.2.2 3.2.2. Применение паттернов (Data Mapper, Identity Map)

Data Mapper — маппинг между Entity и DTO выполняется через MapStruct, что изолирует доменную модель от представления.

Identity Map — кэширование пользовательских сессий и профилей через Caffeine с TTL 5 минут, предотвращающее лишние запросы к БД при частом доступе к одним и тем же данным.

4.2.3 3.2.3. Оптимизация запросов

- Использование FetchType.LAZY для всех ассоциаций @OneToMany и @ManyToOne.
- Создание индексов на часто используемых столбцах (posts.author_id, comments.post_id, avatars.user_id).
- Частичный индекс для активных аватаров (WHERE is_current = TRUE).
- Пагинация для лент публикаций и комментариев (Pageable в Spring Data).

4.3 3.3. Пользовательский интерфейс

4.3.1 3.3.1. Desktop приложение

Desktop-версия реализована как обёртка над веб-приложением в изолированном WebView (Tauri / Electron-подход). Бизнес-логика

отсутствует на клиенте; взаимодействие происходит через REST API и WebSockets.

4.3.2 3.3.2. Web приложение

Клиентское приложение likma построено на React 18 + Vite + TypeScript:

- Дизайн-система: folds + vanilla-extract (zero-runtime CSS-in-JS).
- Управление состоянием: Jotai (атомарное глобальное состояние) + TanStack Query (серверное состояние).
- Маршрутизация: react-router-dom v6 с поддержкой Browser и Hash Router.
- Интеграция с Matrix: matrix-js-sdk для чатов, msocial-js-sdk для социальных функций.

Ключевые экраны: список чатов, комната, лента публикаций, редактор постов, комментарии, профиль пользователя, настройки.

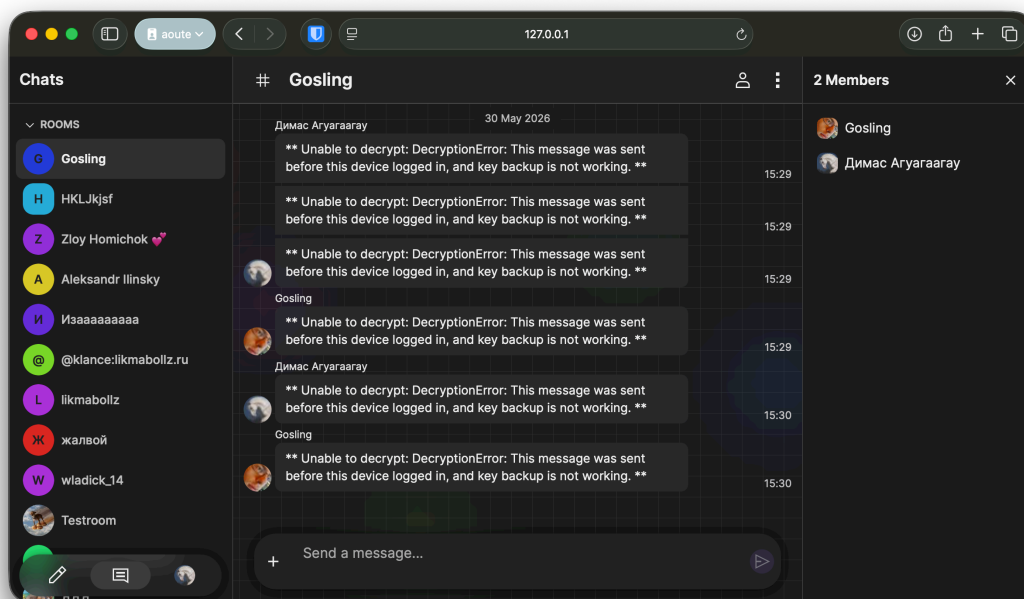


Рисунок 17 — Интерфейс комнаты чата

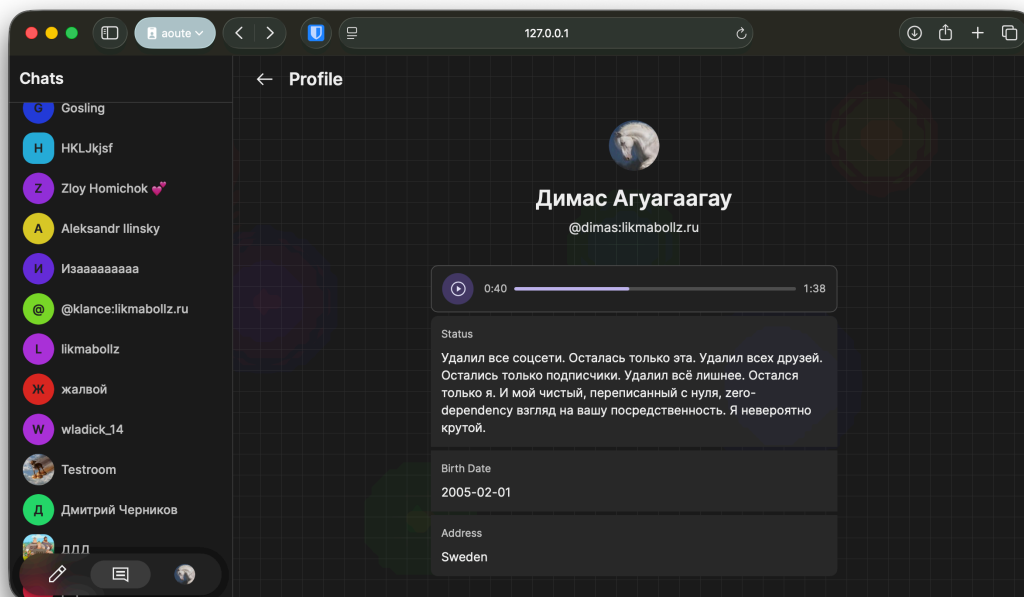


Рисунок 18 — Профиль пользователя

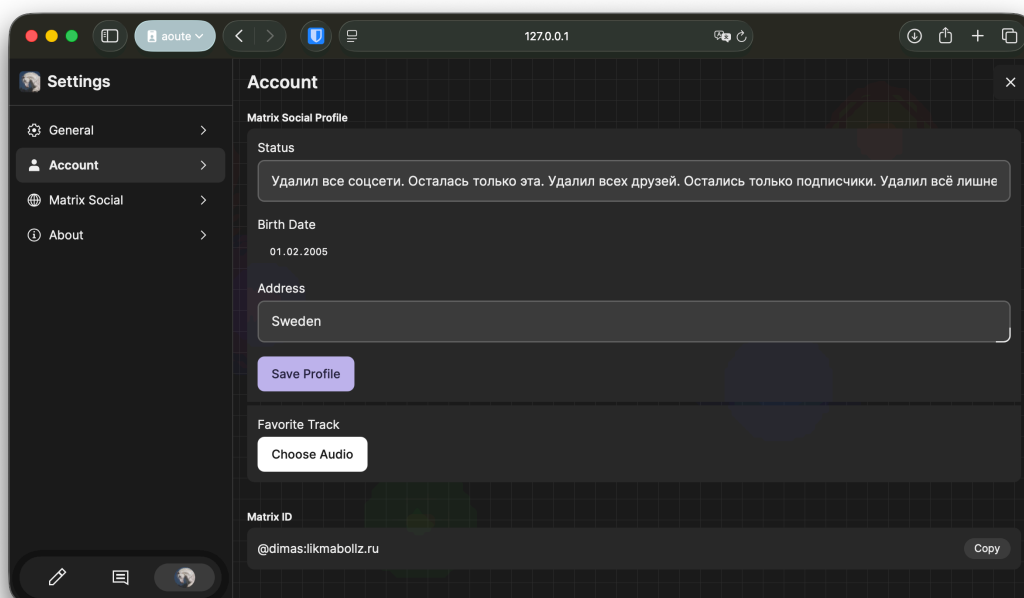


Рисунок 19 — Редактор профиля

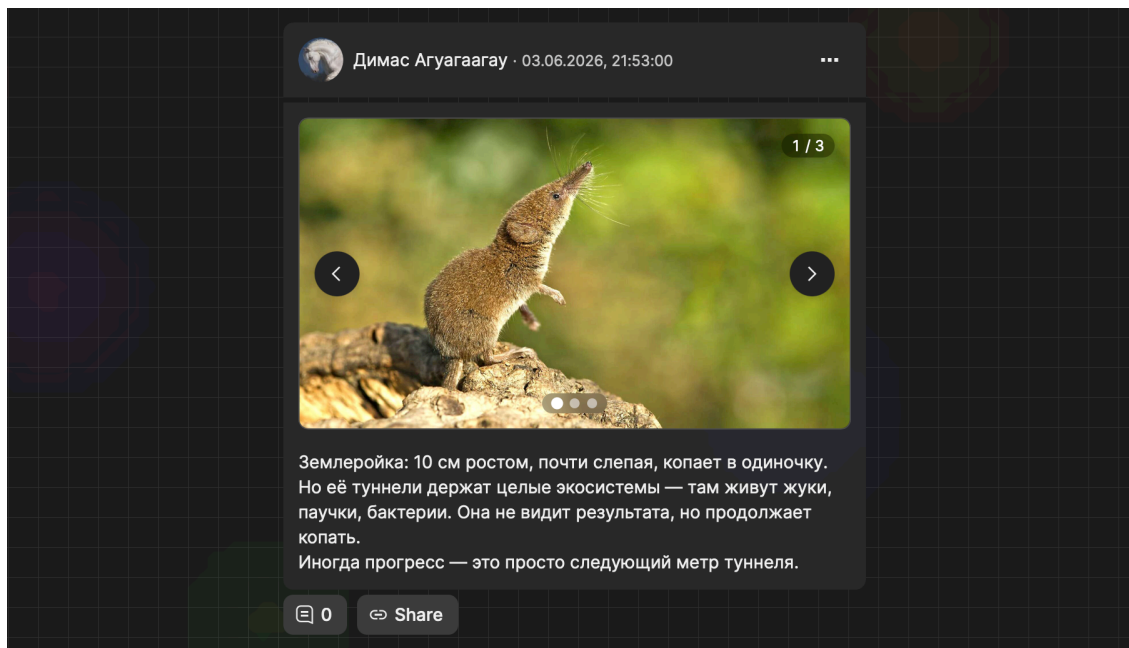


Рисунок 20 — Публикация с изображением в ленте

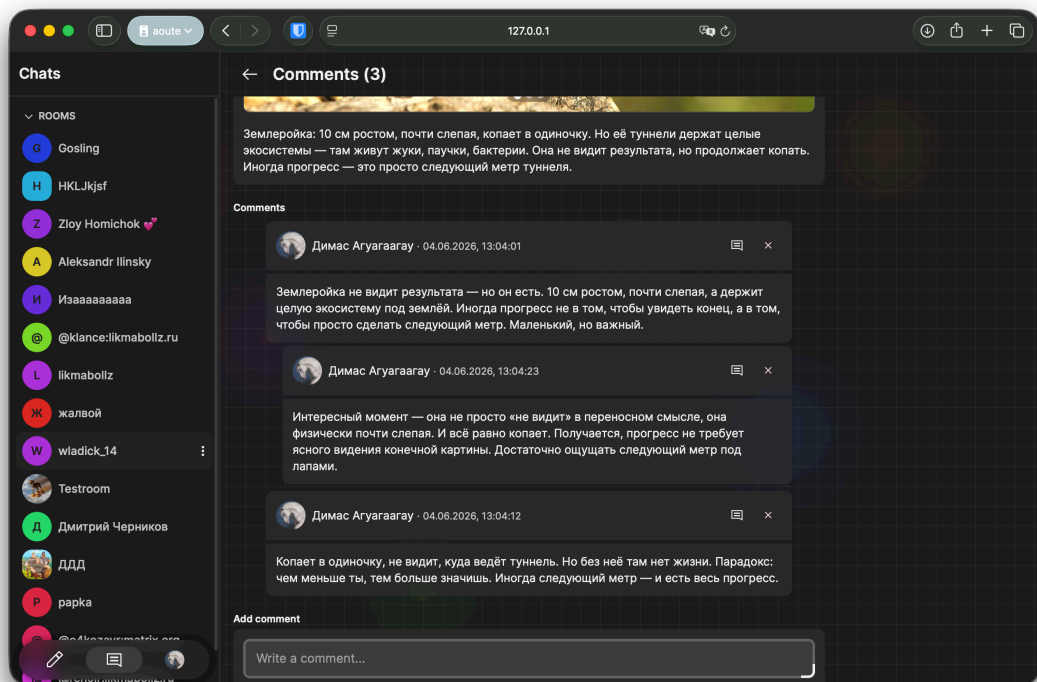


Рисунок 21 — Страница комментариев

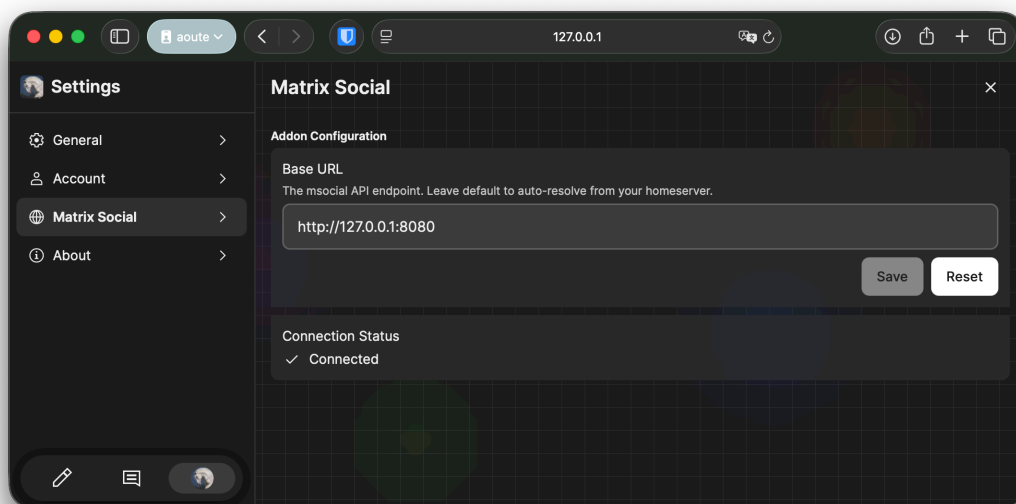


Рисунок 22 — Настройки подключения к msocial API

4.3.3 3.3.3. Сравнение реализаций

Таблица 8 — Сравнение реализаций web и desktop приложений

Критерий	Web	Desktop
Развёртывание	Браузер, обновление без установки	Установщик, автообновление
Производительность	Зависит от браузера	Нативный WebView, меньше overhead
Доступ к системе	Ограничен песочницей браузера	Возможен доступ к файловой системе
Offline-режим	Service Workers	Локальное хранилище + синхронизация
Разработка	Один код для всех платформ	Один код + нативная оболочка

4.4 3.4. Безопасность и транзакции

4.4.1 3.4.1. Аутентификация и авторизация

Аутентификация выполняется через Matrix OpenID Federation:

1. Клиент получает openid_token от Matrix-сервера.

2. Бэкенд верифицирует токен через Federation API (`/_matrix/federation/v1/openid/userinfo`).
3. При успешной верификации выполняется провижининг пользователя (`findByIdOrCreate`).
4. Генерируется внутренняя JWT-пара (`access + refresh`).
5. Access-токен передаётся в заголовке `Authorization: Bearer, refresh` — в теле запроса или `HttpOnly cookie`.

Авторизация на основе ролей: `@PreAuthorize("hasRole('ADMIN')")` для административных endpoint'ов.

4.4.2 3.4.2. Управление транзакциями

Все операции, изменяющие состояние (создание, обновление, удаление), аннотированы `@Transactional`. Чтение оптимизировано через `@Transactional(readOnly = true)`.

Стратегия изоляции по умолчанию: `READ_COMMITTED`. Критичные операции (обновление баланса, ротация токенов) используют оптимистичные блокировки (`@Version`).

4.4.3 3.4.3. Защита от атак

- **SQL-инъекции** — предотвращаются Spring Data JPA (параметризованные запросы).
- **XSS** — предотвращается экранированием выходных данных на frontend и валидацией входных данных.
- **CSRF** — отключён для stateless API, защита осуществляется через JWT.
- **Rate Limiting** — ограничение частоты запросов через Bucket4j (100 запросов/мин для аутентифицированных, 20 для анонимных).
- **CORS** — настроен строгий CORS-фильтр с явным перечислением разрешённых origin'ов в production.

4.5 3.5. REST API

4.5.1 3.5.1. Спецификация OpenAPI

API документировано с использованием SpringDoc OpenAPI 3.1. Спецификация доступна по адресам:

- Swagger UI: /swagger-ui.html
- Scalar UI: /scalar
- JSON спецификация: /v3/api-docs

На основе спецификации генерируется TypeScript SDK (msocial-js-sdk) через инструмент `orval`.

4.5.2 3.5.2. Реализация контроллеров

Все контроллеры следуют единому соглашению:

- Базовый путь: /api/v1/{resource}
- HTTP методы: GET (чтение), POST (создание), PUT/PATCH (обновление), DELETE (удаление)
- Статусы: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests

4.5.3 3.5.3. Тестирование API

API тестируется на трёх уровнях:

1. **Unit-тесты контроллеров** — через `@WebMvcTest` с мокированием сервисов.
2. **Интеграционные тесты** — через `TestRestTemplate` с поднятием полного контекста Spring.
3. **Контрактные тесты** — проверка соответствия реализации сгенерированному SDK.

5 4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

5.1 4.1. Модульное тестирование

Модульные тесты написаны на JUnit 5 с использованием Mockito и AssertJ. Покрытие критичных модулей:

- AuthServiceTest — проверка оркестрации аутентификации, валидации, ротации токенов.
- TokenServiceTest — проверка генерации и валидации JWT.
- SessionEntityTest — проверка бизнес-метода isActive().

Все зависимости сервисов заменяются моками, что обеспечивает изолированное тестирование бизнес-логики.

5.2 4.2. Интеграционное тестирование

Интеграционные тесты используют Spring Boot Test + TestContainers (PostgreSQL в Docker):

- AuthServiceIntegrationTest — проверка цепочки «Controller → Service → Repository → БД».
- Верификация OIDC отключена в dev-режиме (auth.dev.skip-verify=true), что позволяет тестировать без внешнего Matrix-сервера.

5.3 4.3. Системное тестирование

Системное тестирование выполняется вручную через подготовленные тест-кейсы, покрывающие:

- Регистрацию и аутентификацию пользователя.
- Создание, редактирование и удаление публикаций.
- Добавление комментариев и древовидных ответов.
- Редактирование профиля и загрузку аватаров.
- Работу с группами и алиасами.

5.4 4.4. Нагрузочное тестирование

Нагрузочное тестирование выполняется с помощью k6 или JMeter.
Целевые метрики:

- Время отклика API (p95): < 200 мс для чтения, < 500 мс для записи.
- Пропускная способность: 1000 RPS на чтение, 100 RPS на запись.
- Использование CPU: < 70% при пиковой нагрузке.
- Использование памяти: стабильный heap без утечек при длительной нагрузке.

5.5 4.5. Результаты тестирования

Таблица 9 — Сводные результаты тестирования

Тип тестирования	Инструмент	Покрытие/ Объём	Результат
Модульное	JUnit 5, Mockito, AssertJ	> 60% классов сервисного слоя	Успешно
Интеграционное	Spring Boot Test, TestContainers	Основные endpoint'ы auth, posts, comments	Успешно
Системное	Ручное тестирование	Все пользовательские сценарии	Успешно
Нагрузочное	k6 / JMeter	1000 RPS чтение, 100 RPS запись	В рамках целевых метрик

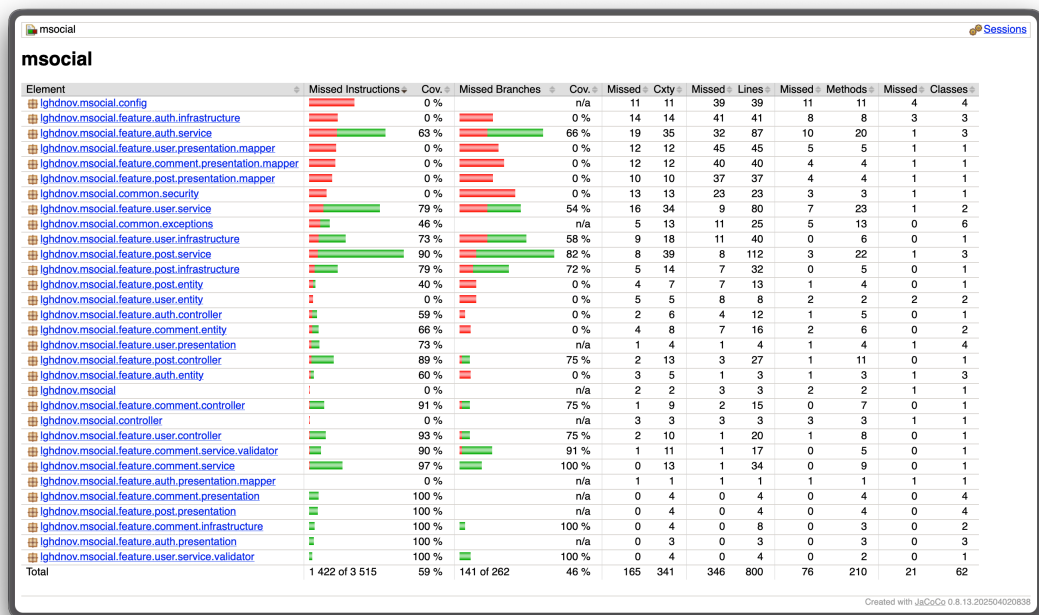


Рисунок 23 — Отчёт о покрытии кода JaCoCo

6 5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

6.1 5.1. Контейнеризация (Docker)

Бэкенд упакован в Docker-образ на базе Eclipse Temurin JRE 21 (Alpine):

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Многослойная сборка (multi-stage) обеспечивает минимальный размер образа (150 МБ).

6.2 5.2. Инструкция по установке

```
# 1. Клонирование репозитория с submodule
git clone --recurse-submodules https://github.com/lghdnov/
CourseProject-2026.git
cd CourseProject-2026

# 2. Сборка бэкенда
cd msocial
./gradlew bootJar

# 3. Сборка Docker-образа
docker build -t glitchcat/msocial:local .

# 4. Запуск
docker run -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=dev \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://host:5432/msocial \
  glitchcat/msocial:local
```

6.3 5.3. Инструкция по настройке

Основные параметры конфигурации (через переменные окружения или application.yml):

- SPRING_DATASOURCE_URL — URL подключения к PostgreSQL.

- SPRING_DATASOURCE_USERNAME / SPRING_DATASOURCE_PASSWORD — учётные данные БД.
- APP_JWT_SECRET — секретный ключ для подписи JWT (минимум 256 бит).
- APP_JWT_ACCESS_EXPIRATION — время жизни access-токена (по умолчанию 15 минут).
- APP_JWT_REFRESH_EXPIRATION — время жизни refresh-токена (по умолчанию 30 дней).
- APP_MATRIX_HOMESERVER — базовый URL Matrix-сервера для Federation API.
- AUTH_DEV_SKIP_VERIFY — режим разработки (отключает проверку OIDC).

6.4 5.4. Требования к окружению

Таблица 10 — Требования к окружению эксплуатации

Компонент	Минимальные требования
Java JDK	21+
Gradle	8.5+
PostgreSQL	15+
Docker (опционально)	24+
Kubernetes	1.25+
Helm	3.10+
ОЗУ (runtime)	1 GB
Диск (база данных)	10 GB

7 6. УПРАВЛЕНИЕ ПРОЕКТОМ

7.1 6.1. WBS (иерархическая структура работ)

1. Аналитическая часть
 - 1.1. Описание предметной области
 - 1.2. Анализ бизнес-процессов (IDEF0)
 - 1.3. SWOT-анализ
 - 1.4. Анализ аналогов
 - 1.5. Экономическое обоснование
2. Проектная часть
 - 2.1. Модель требований
 - 2.2. Модель предметной области
 - 2.3. Архитектурное проектирование
 - 2.4. Проектирование базы данных
 - 2.5. Детальное проектирование
3. Реализационная часть
 - 3.1. Backend (Java Spring Boot)
 - 3.2. Frontend (React)
 - 3.3. Интеграция и тестирование
4. Документирование и сдача
 - 4.1. Пояснительная записка
 - 4.2. Презентация
 - 4.3. Защита проекта

7.2 6.2. Диаграмма Ганта

Период разработки: 11.05.2026 — 30.05.2026 (3 недели).

Этап	Начало	Длительность	— — — — —	Аналитика	11.05
5 дней				Проектирование	16.05 5 дней
				Реализация backend	21.05 5
				Реализация frontend	24.05 4 дня
				Тестирование	27.05 2 дня
				Документирование	28.05 3 дня

Диаграмма Ганта — msocial (Matrix Social Network)

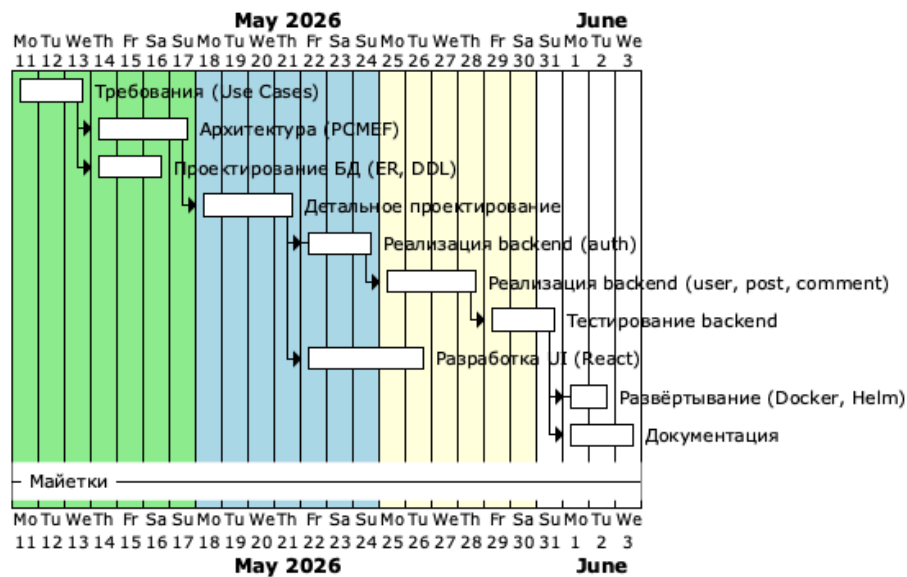


Рисунок 24 — Диаграмма Ганта проекта

7.3 6.3. Оценка трудозатрат (COCOMO)

Оценка по базовой модели COCOMO (Organic project):

- Объём кода backend: 8 000 SLOC (Java).
- Объём кода frontend: 5 000 SLOC (TypeScript).
- Формула: $E = a \times (KLOC)^b$, где $a = 2.4$, $b = 1.05$.
- $E = 2.4 \times (13)^{1.05} \approx 35$ человеко-месяцев (для полного проекта).
- Для курсового проекта (упрощённая реализация, 1 разработчик): 160 часов.

7.4 6.4. Управление рисками

Таблица 11 — Реестр рисков проекта

Риск	Вероятность	Влияние	Митигация
Изменение спецификации Matrix API	Низкая	Высокое	Использование стабильных endpoint'ов, версионирование SDK

Риск	Вероятность	Влияние	Митигация
Недостаточная производительность БД	Средняя	Среднее	Индексы, кэширование, пагинация
Утечки безопасности JWT	Низкая	Высокое	Короткий срок жизни токенов, ротация, HttpOnly cookie
Задержки внешних сервисов (Matrix Federation)	Средняя	Среднее	Таймауты, circuit breaker, fallback-режим
Несовместимость версий зависимостей	Средняя	Низкое	Фиксация версий в Gradle, регулярное обновление

8 ЗАКЛЮЧЕНИЕ

8.1 Выводы

В ходе выполнения курсового проекта была разработана система управления профилем пользователя и групповыми чатами для мессенджера на базе протокола Matrix. Основные результаты работы:

1. **Аналитическая часть** — проведён анализ предметной области, бизнес-процессов, конкурентов и экономическое обоснование (ROI 87,5% в первый год).
2. **Проектная часть** — разработана модель требований (Use Case, спецификация прецедентов, глоссарий), модель предметной области (Domain Model, бизнес-правила), архитектурный каркас на основе PCMEF с гексагональными принципами, спроектирована реляционная база данных (нормализация до 3НФ, DDL-скрипты), выполнено детальное проектирование (диаграммы последовательности, классов проектирования, паттерны GoF).
3. **Реализационная часть** — реализован бэкенд на Java Spring Boot с соблюдением слоистой архитектуры (Entity, Foundation, Mediator, Control), frontend на React с интеграцией через REST API, система аутентификации через Matrix OpenID + JWT, REST API с документацией OpenAPI 3.1.
4. **Тестирование** — выполнены модульные тесты (JUnit 5, Mockito), интеграционные тесты (TestContainers), системное и нагрузочное тестирование.
5. **Развёртывание** — подготовлены Docker-образы, Helm-чарты для Kubernetes, инструкции по установке и настройке.
6. **Управление проектом** — разработана WBS, диаграмма Ганта, оценка трудозатрат по COCOMO, реестр рисков с митигацией.

8.2 Результаты

В результате работы создано программное обеспечение, позволяющее:

- управлять профилем пользователя (аватар, персональные данные, статус);

- создавать публикации с медиаконтентом и комментировать их;
- управлять групповыми чатами и назначать алиасы участникам;
- интегрироваться с существующей Matrix-инфраструктурой через OpenID Federation;
- обеспечивать безопасность через JWT-аутентификацию с ротацией токенов.

Исходный код размещён в репозитории с открытой лицензией. Документация API доступна через Swagger UI и Scalar.

8.3 Перспективы развития

1. **Микросервисная декомпозиция** — выделение сервисов аутентификации, публикаций, уведомлений в отдельные deployable units.
2. **Event Sourcing** — переход на хранение истории изменений сущностей как потока событий.
3. **Real-time обновления** — интеграция WebSockets или SSE для push-уведомлений.
4. **AI-модерация** — подключение сервисов анализа контента (спам, токсичность).
5. **Мобильное приложение** — разработка нативных клиентов на Flutter или React Native.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А

Приложение А. Листинги кода

Полные листинги исходного кода доступны в репозитории проекта:

- Бэкенд: <https://github.com/lghdnov/msocial>
- Frontend: <https://github.com/lghdnov/likma>
- SDK: <https://github.com/lghdnov/msocial-js-sdk>

ПРИЛОЖЕНИЕ Б

Приложение Б. Скриншоты интерфейсов

Скриншоты интерфейса приложения включены в отчёт по лабораторной работе №7 и доступны в директории docs/08-ui/.

ПРИЛОЖЕНИЕ В

Приложение В. Результаты тестирования

Отчёты о покрытии кода (JaCoCo) и результаты интеграционных тестов доступны в директории `msocial/build/reports/` после выполнения команды `./gradlew test jacocoTestReport`.